Ray-tracing

Dr. Sk. Md. Masudul Ahsan
Professor
Dept. of CSE, KUET

Ray Tracing

- ~ a global illumination model
- Integrates the task of
  - Shading
  - shadow generation
  - Hidden surface removal
  - Projection
  - Scan conversion
- Ray tracing is a powerful rendering technique that is the foundation of many modern photoreal rendering algorithms
- The original ray tracing technique was proposed in 1980 by Turner Whitted, although there were suggestions about the possibility in scientific papers dating back to 1968

What is ray tracing

- Ray tracing is a technique for rendering 3D graphics with very complex light interactions.
  - You can create pictures full of mirrors, transparent surfaces, and shadows, with stunning results.
- A very simple method to understand and implement.
- It is based on the idea that you can model reflection and refraction by recursively following the path that light takes as it bounces through an environment

Ray-Tracing Examples



https://deepubalan.com/blog/2010/01/03/pov-ray-the-power-of-ray-tracing/

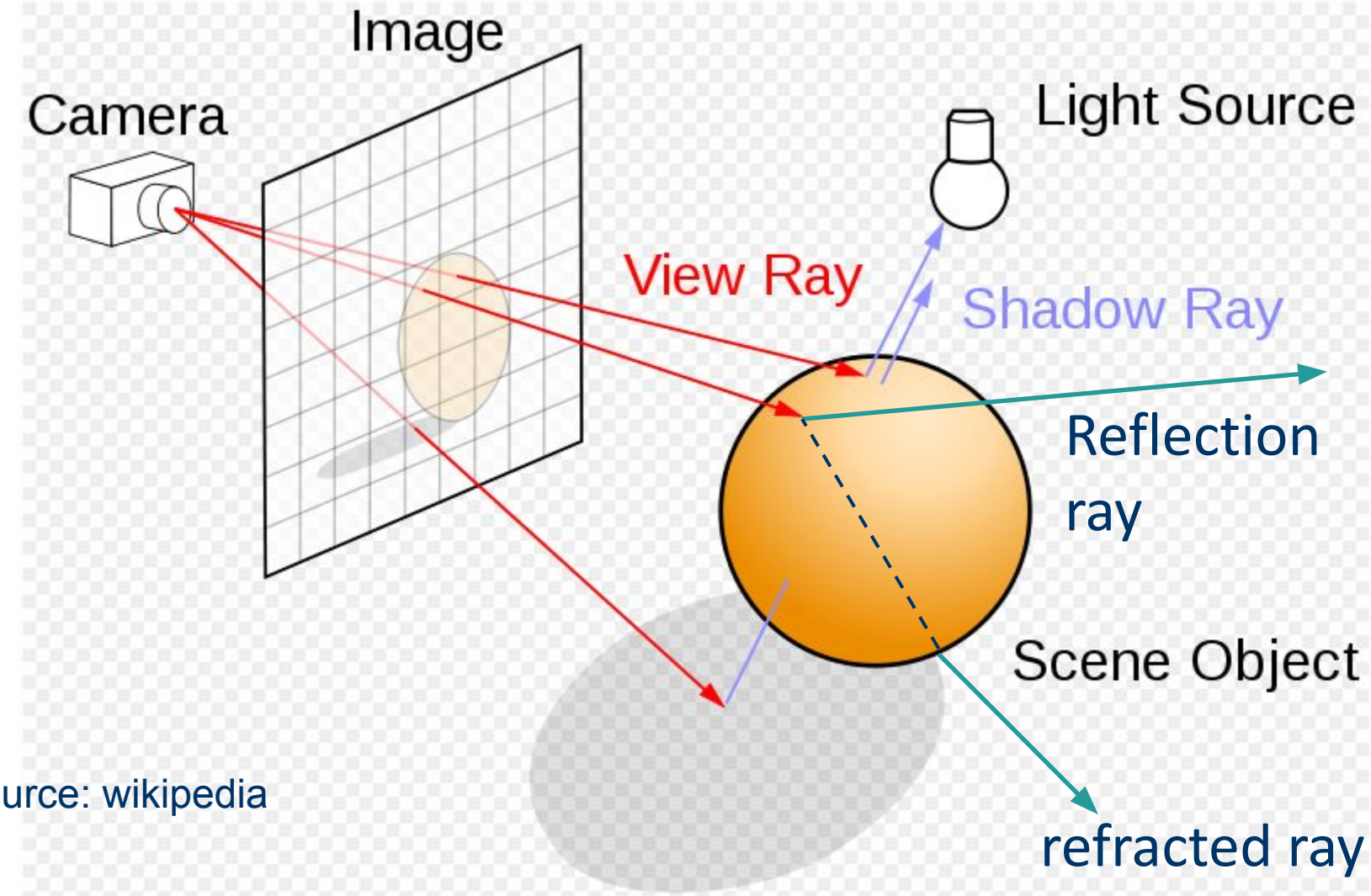
Ein mit POV-Ray erzeugtes Bild.
© Gilles Tran www.oyonale.com)

Ray-Tracing Examples



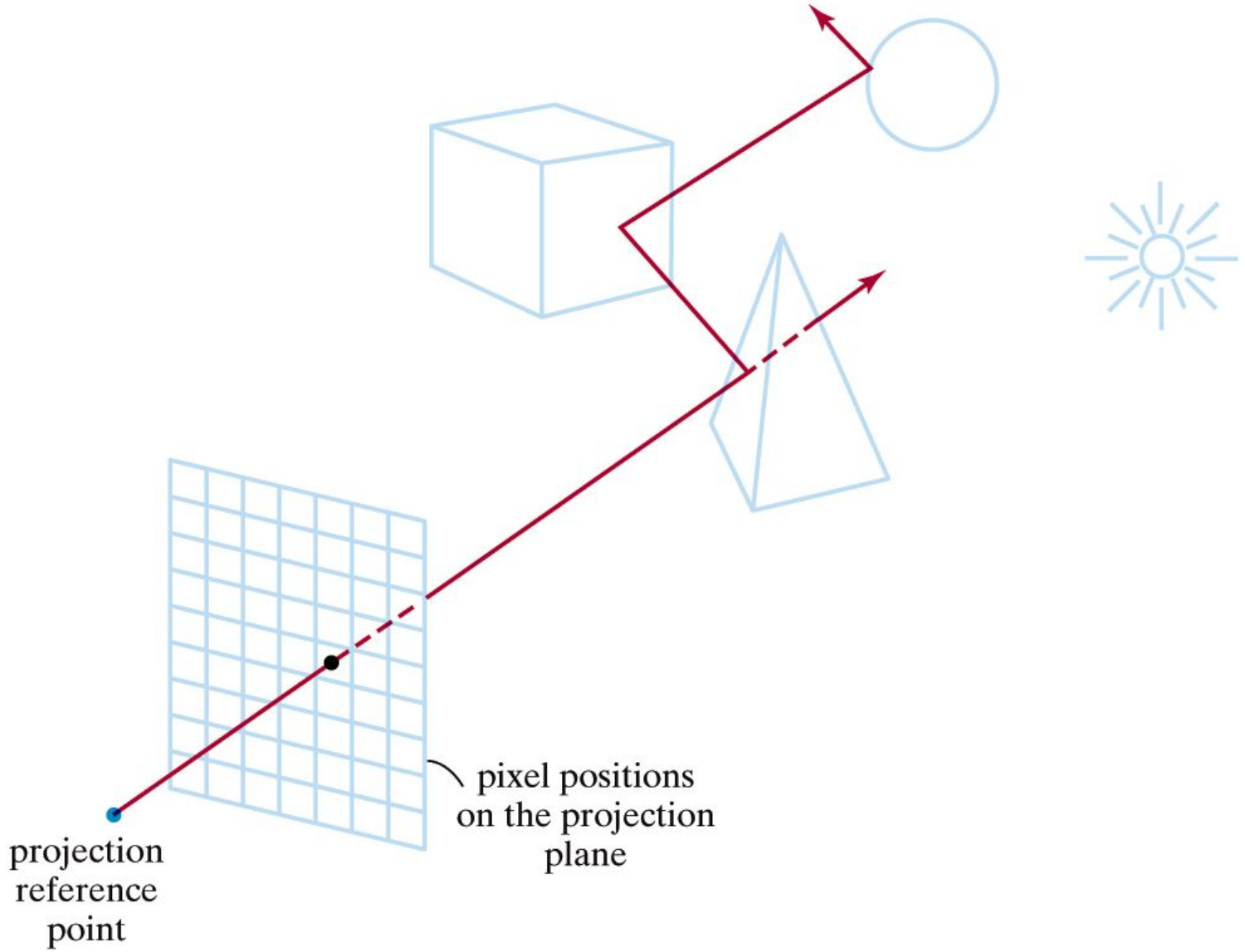
Kettle, Mike Miller, POV-Ray

Ray-Tracing Setup

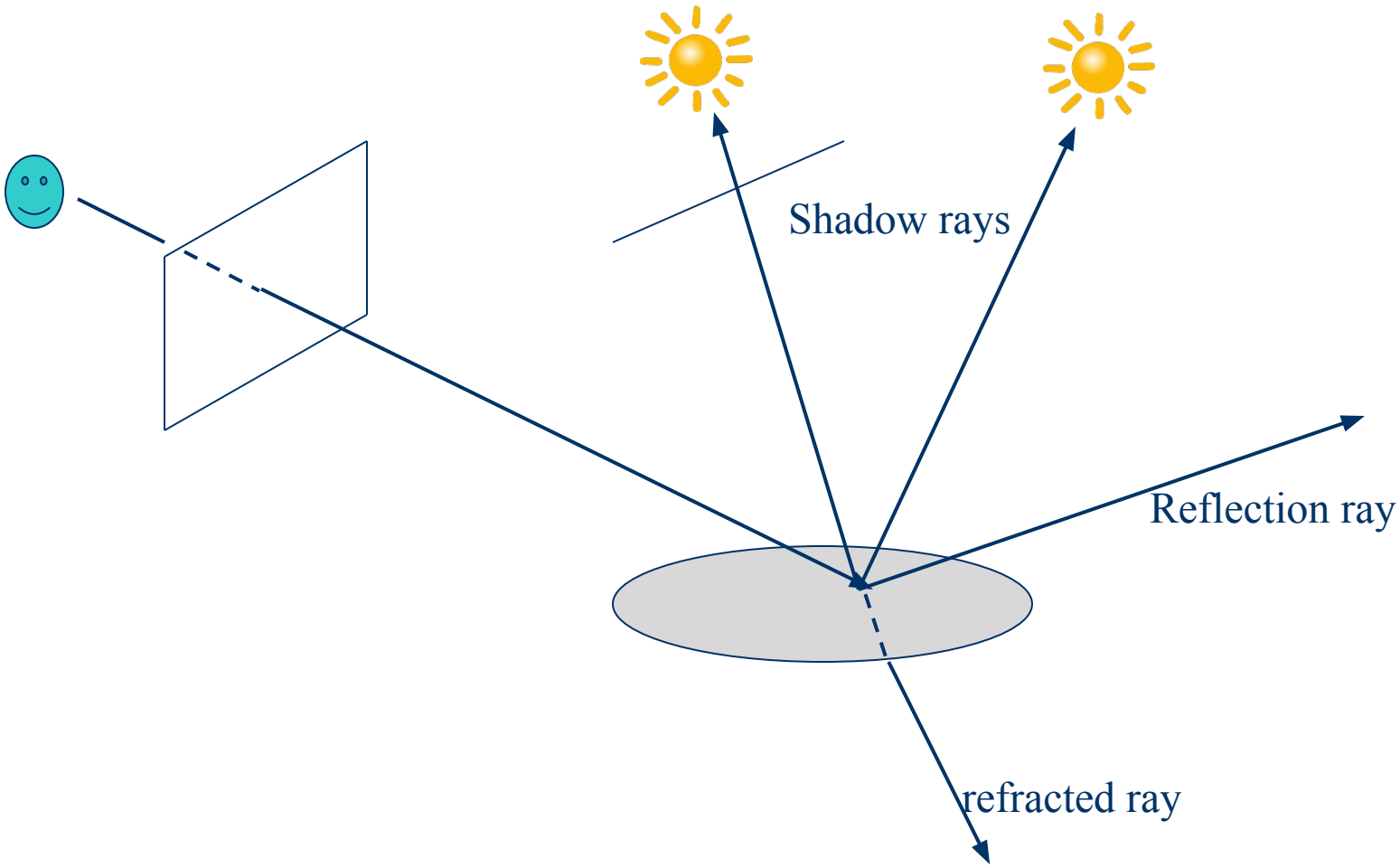


Source: wikipedia

Ray-Tracing Setup



Ray-Tracing Setup



Ray Tracing

- Classic ray tracing shoots virtual view rays into the scene from the camera and traces their paths as they bounce around
- With ray tracing, one can achieve a wide variety of complex lighting effects, such as accurate shadows and reflections/refractions from curved surfaces
- Achieving these effects with the same precision is difficult if not impossible with a more traditional rendering pipeline
- Ray tracing offers a big advance in visual quality, but comes with an expensive price of notoriously slow rendering times

Ray Intersections

- Tracing a single ray requires determining if that ray intersects any one of potentially millions of primitives
  - This is the basic problem of *ray intersection*
- Many algorithms exist to make this not only feasible, but remarkably efficient
- Tracing one ray is a complex problem and requires serious work to make it run at an acceptable speed
- Of course, the big problem is the fact that one needs to trace lots of rays to generate a high quality image

Ray-Tracing Process

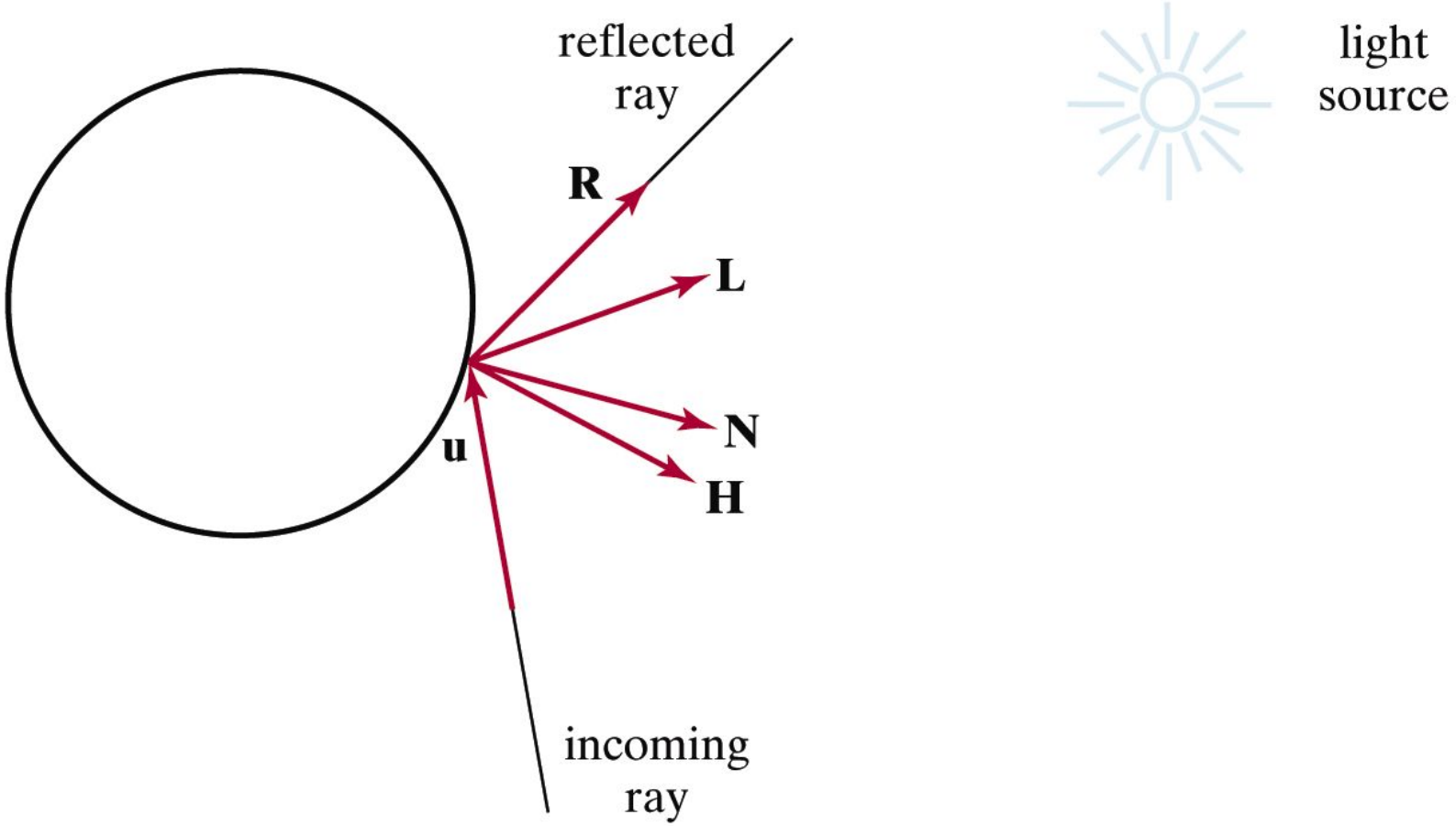
- Builds the image pixel by pixel
- Fire a single ray for each pixel position into the scene along the projection path
  - If the ray doesn't hit anything, then we can color the pixel to some specified 'background' color
  - Otherwise,
    - Determine which surfaces the ray intersects and order these by distance from the pixel
      - it is possible that the ray will hit several surfaces, but we only care about the **closest one** to the camera
      - The nearest surface to the pixel is the visible surface for that pixel
    - Shoot rays toward each light. If they hit something, the object is shadowed from that light, otherwise use "standard model" for the light
    - Reflect a ray off the visible surface along the specular reflection angle
    - For transparent surfaces also send a ray through the surface in the refraction direction
    - Repeat the process for these secondary rays

Terminating Ray-Tracing

- We terminate a ray-tracing path when any one of the following conditions is satisfied:
  - The ray intersects no surfaces
  - The ray intersects a light source that is not a reflecting surface
  - A maximum allowable number of reflections have taken place

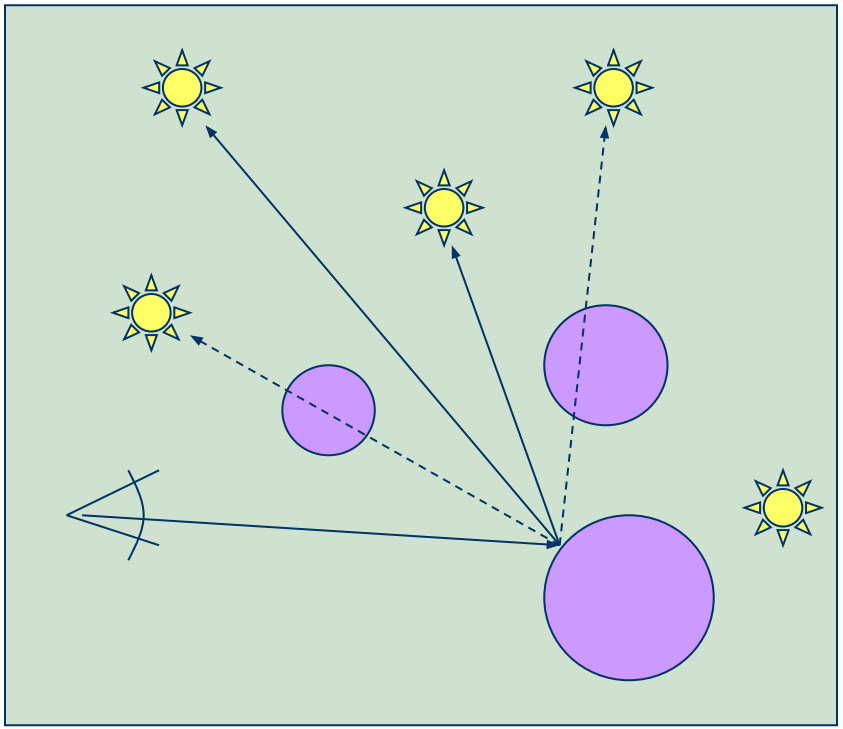
Ray-Tracing & Illumination Models

- At each surface intersection the illumination model is invoked to determine the surface intensity contribution



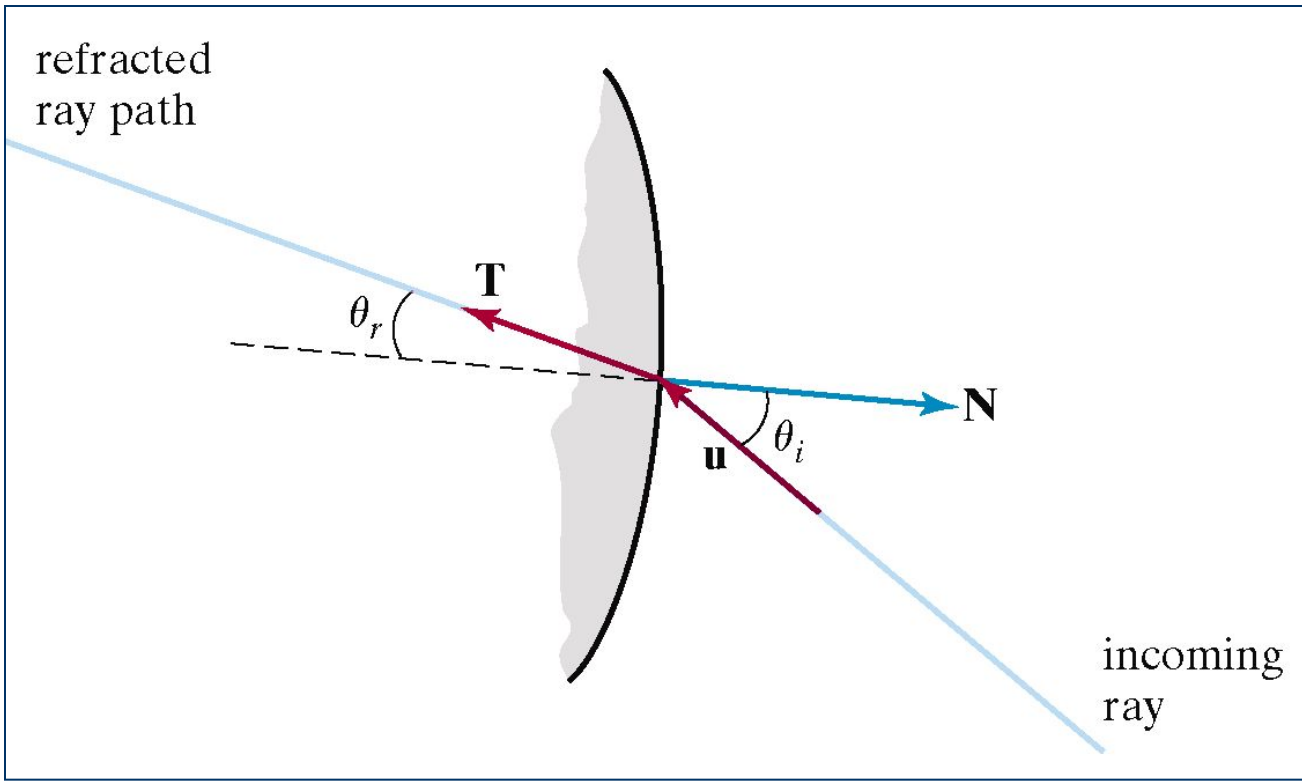
The Shadow Ray

- The path from the intersection to the light source is known as the **shadow ray**
- If any object intersects the shadow ray between the surface and the light source then the surface is in shadow with respect to that source



Ray-Tracing & Transparent Surfaces

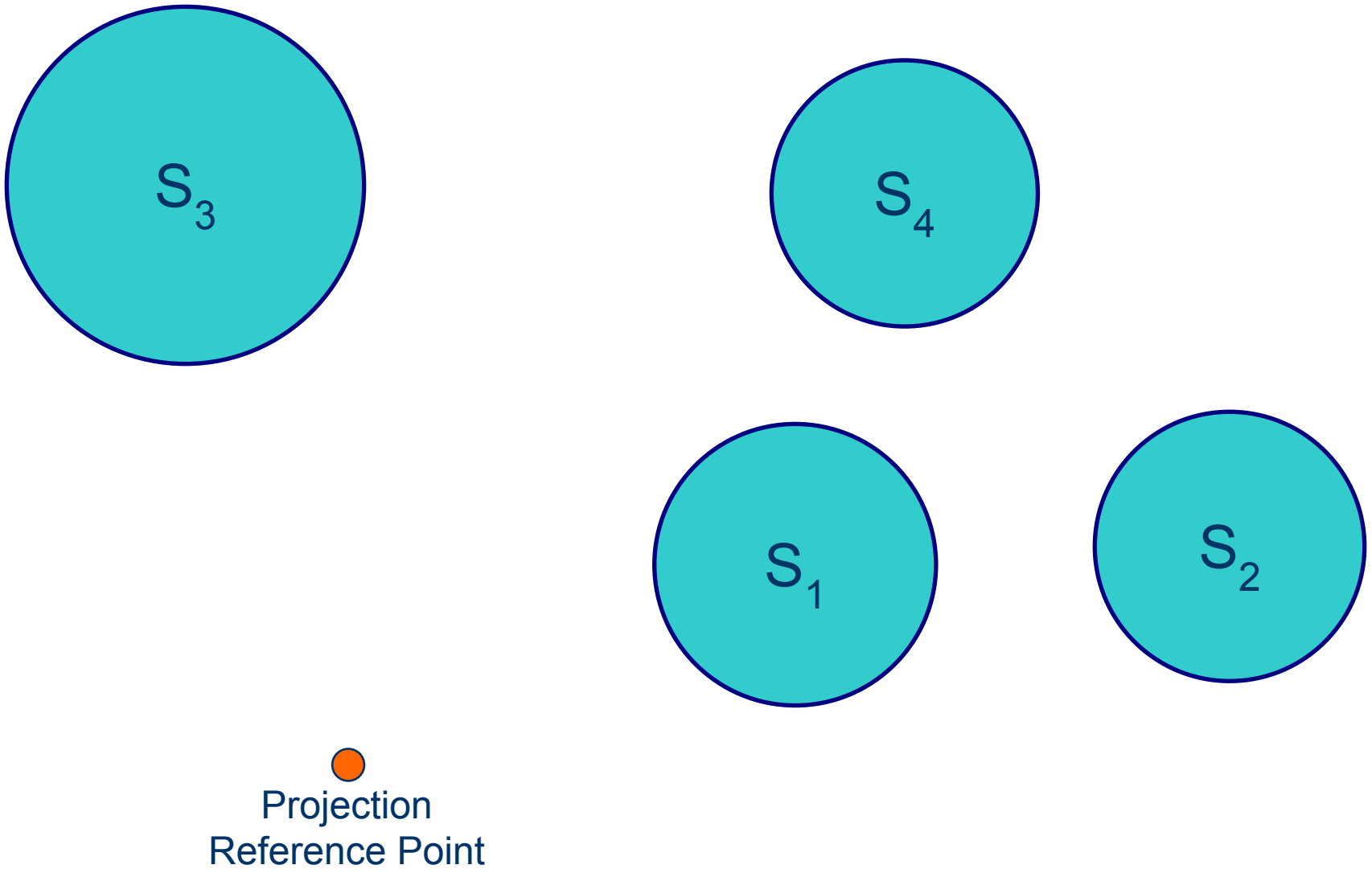
- For transparent surfaces we need to calculate a ray to represent the light refracted through the material
- The direction of the refracted ray is determined by the refractive index of the material



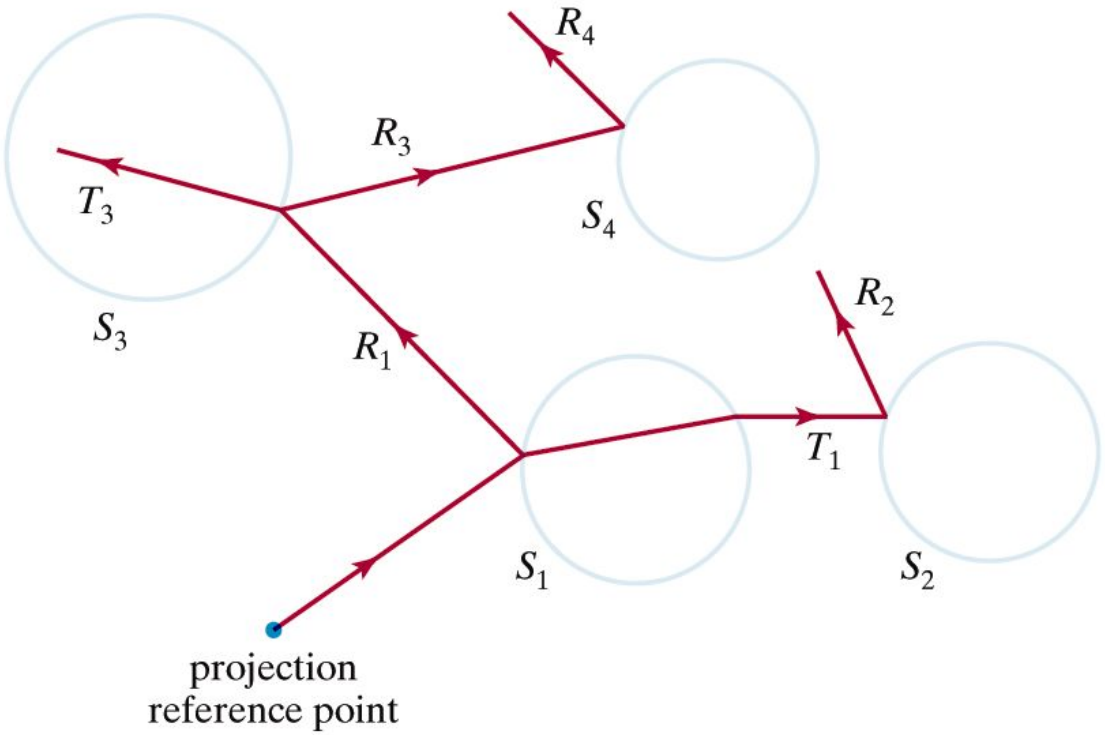
Ray-Tracing Tree

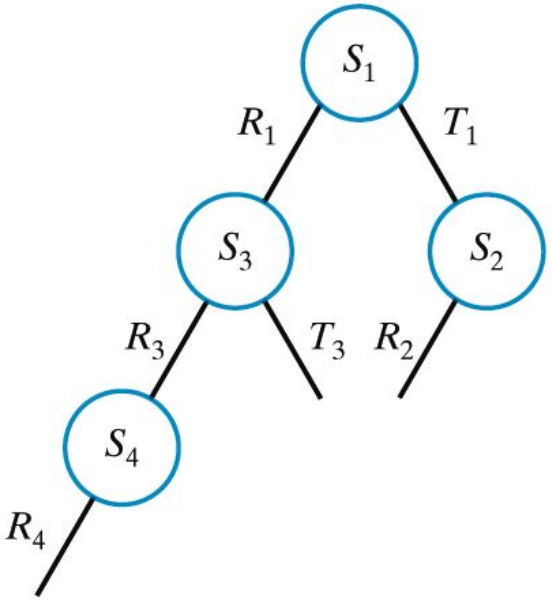
- As the rays ricochet around the scene each intersected surface is added to a binary **ray-tracing tree**
  - The left branches in the tree are used to represent reflection paths
  - The right branches in the tree are used to represent transmission paths
- The tree's nodes store the intensity at that surface
- The tree is used to keep track of all contributions to a given pixel

Ray-Tracing Tree Example



Ray-Tracing Tree Example (cont…)





Ray-Tracing Tree (cont…)

- After the ray-tracing tree has been completed for a pixel the intensity contributions are accumulated
- We start at the terminal nodes (bottom) of the tree
- The surface intensity at each node is attenuated by the distance from the parent surface and added to the intensity of the parent surface
- The sum of the attenuated intensities at the root node is assigned to the pixel

Recursive Ray Tracer

- Use the camera analogy
- Determine light rays responsible for pixels, in opposite direction (Primary ray)
- If primary ray doesn't intersect any object, corresponding pixel is simply set to Bg color
- If intersects - Secondary rays used for surface shading
  - Local contribution
  - Reflected contribution
  - Transmitted contribution

Implementation - Recursive Ray Tracer

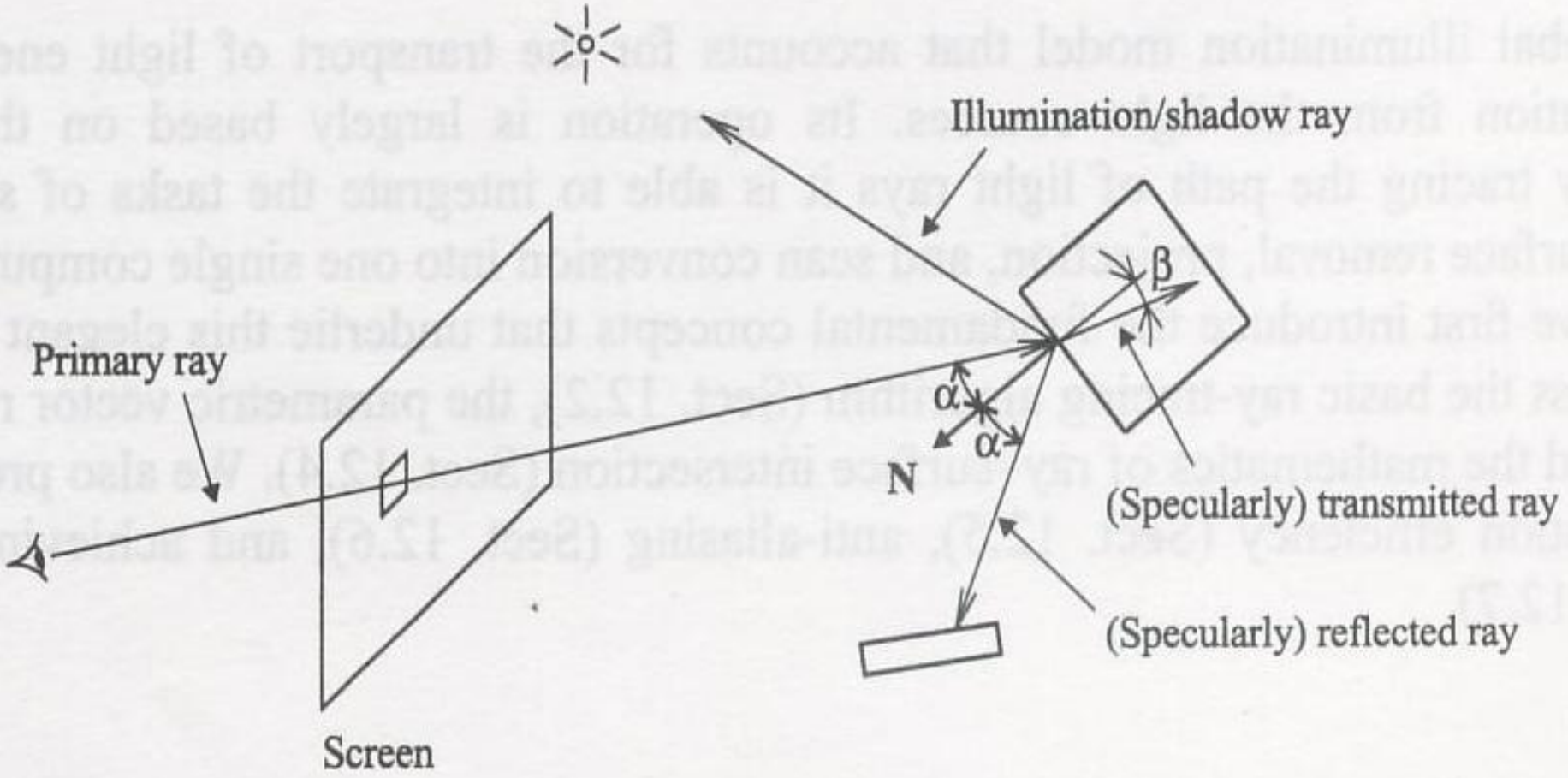


Fig. 12-3 Ray tracing.

Recursive Ray Tracer

- Employ the Phong formula to describe the contribution

$$I = I_a k_a + \sum_{i=1}^{n} I_{p_i} (k_d L_i . N + k_s (R_i . V)^k) + k_r I_r + k_t I_t$$

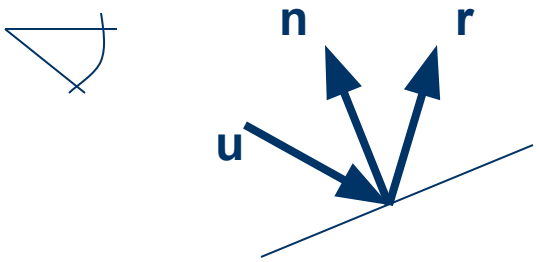
- I_r and I_t are obtained recursively by treating them as primary ray
- I_r is calculated by evaluating the above eq at the closest surface it intersects, as the same as primary ray

Recursive Ray Tracer Algorithm

```
rayTrace(ray, depth, color)
{
    determine closest intersection of ray with an object
    if (no intersection) color = bg
    else
    {
        color = local contribution
        if (depth > 1)
        {
            calculate reflected ray
            rayTrace(reflected ray, depth -1, ref_color)
            calculate transmitted ray
            rayTrace(transmitted ray, depth -1, trans_color)
            Color= color + k_r.ref_color + k_t.trans_color
        }
    }
}
```

Recursive Ray Tracer

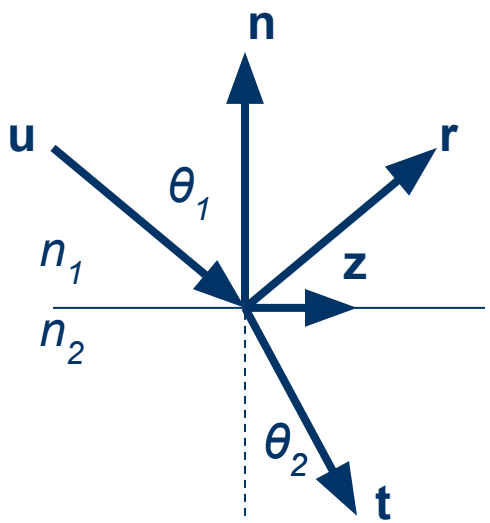
- Local Contribution
  - Direct contribution from light source(s)
  - A shadow ray is sent from the surface point to the light source
  - If the ray is blocked before reaching the light source, the point is in shadow
- Reflected Contribution
  - Refers inter object reflection
  - A reflected ray – reflection of primary ray with surface normal



$$\mathbf{r} = \mathbf{u} - 2(\mathbf{u} \cdot \mathbf{n})\mathbf{n}$$

Recursive Ray Tracer

- Transmitted Contribution
  - Refers the transmission of light energy coming from behind the surface
  - A transmitted ray – refraction of primary ray with surface normal
  - Use Snell's law $\eta_1 \sin\theta_1 = \eta_2 \sin\theta_2$

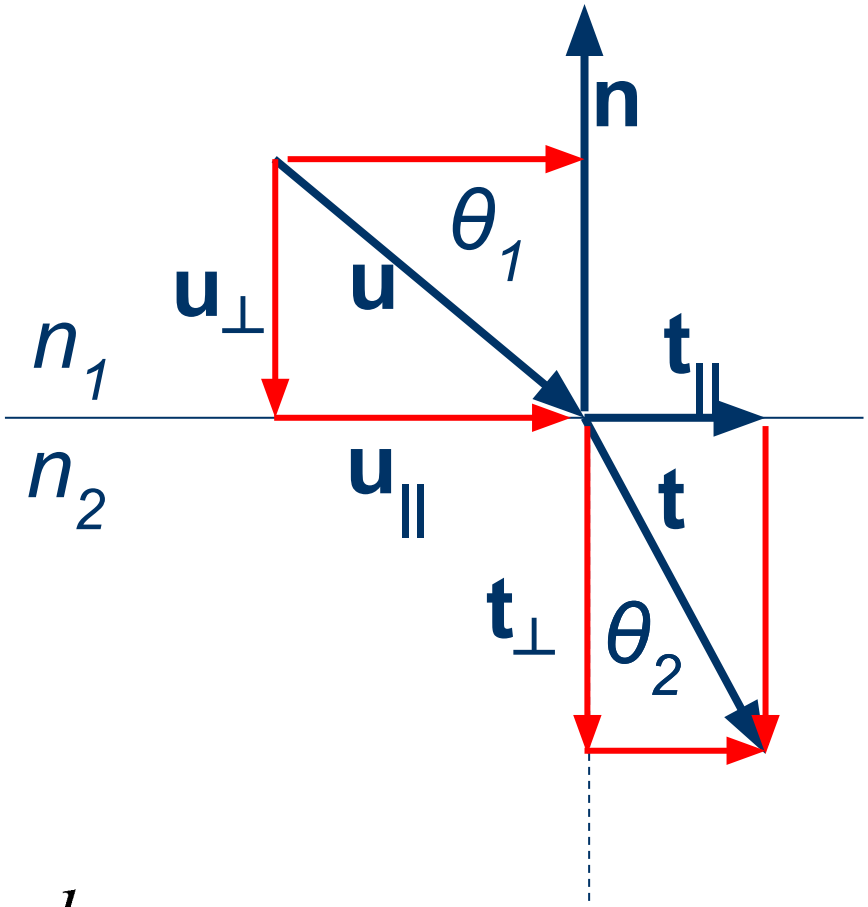


$$\mathbf{r} = \mathbf{u} - 2(\mathbf{u}\cdot\mathbf{n})\mathbf{n}$$

$$\mathbf{z} = \frac{n_1}{n_2}\left(\mathbf{u} - (\mathbf{u}\cdot\mathbf{n})\mathbf{n}\right)$$

$$\mathbf{t} = \mathbf{z} - \left(\sqrt{1 - |\mathbf{z}|^2}\right)\mathbf{n}$$

Transmitted Ray



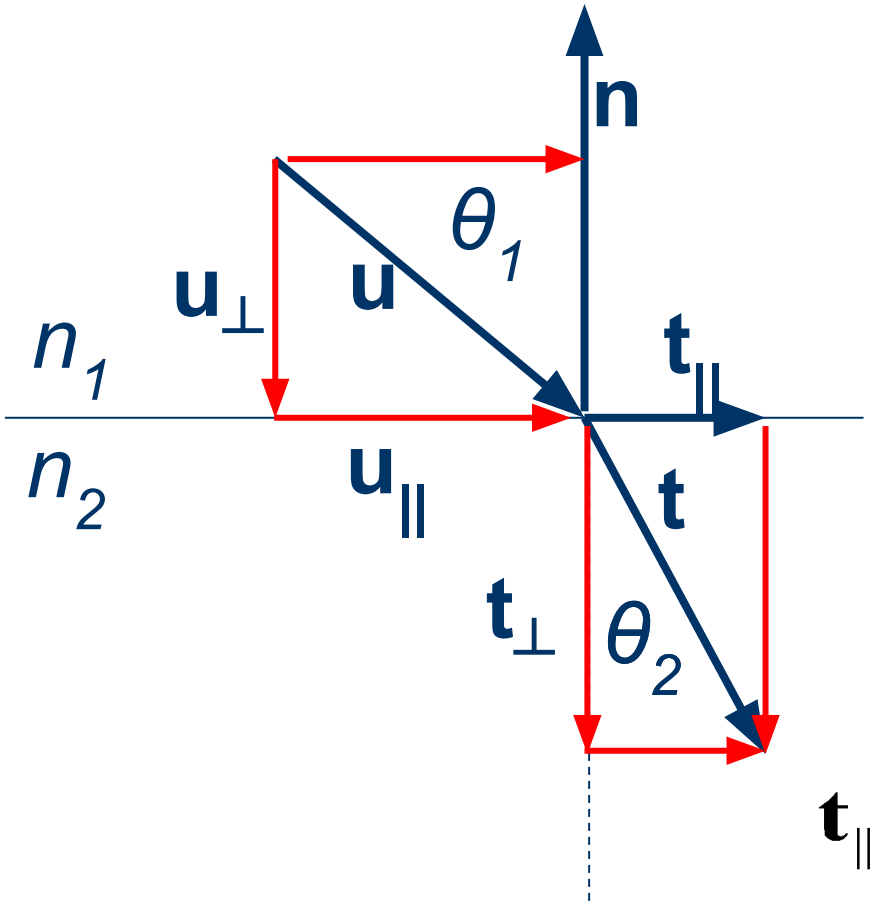
- Let, u, n, t all are unit vector
- $\mathbf{t} = \mathbf{t}_{\perp} + \mathbf{t}_{\parallel}$
- $\mathbf{t}_{\parallel} = \alpha \mathbf{u}_{\parallel}$; $\mathbf{t}_{\perp} = -\beta \mathbf{n}$

here,

$$\mathbf{u} \bullet \mathbf{n} = |\mathbf{u}||\mathbf{n}|\cos(180^\circ - \theta_1)$$
$$= -\cos\theta_1$$

$$\mathbf{u}_{\parallel} = \mathbf{u} - \mathbf{u}_{\perp}$$
$$= \mathbf{u} - (-\gamma\,\mathbf{n})$$
$$= \mathbf{u} + |\mathbf{u}|\cos\theta_1\mathbf{n}$$
$$= \mathbf{u} + \cos\theta_1\mathbf{n}, \text{ since } |\mathbf{u}| = 1$$
$$= \mathbf{u} - (\mathbf{u} \bullet \mathbf{n})\mathbf{n},$$
$$\text{since } \mathbf{u} \bullet \mathbf{n} = -\cos\theta_1$$

Transmitted Ray



$$\eta_1 \sin\theta_1 = \eta_2 \sin\theta_2$$

$$\Rightarrow \eta_1 \frac{|\mathbf{u}_{\parallel}|}{|\mathbf{u}|} = \eta_2 \frac{|\mathbf{t}_{\parallel}|}{|\mathbf{t}|}$$

$$\Rightarrow \eta_1 |\mathbf{u}_{\parallel}| = \eta_2 |\mathbf{t}_{\parallel}|, \text{ since } \mathbf{u},\mathbf{t} = 1$$

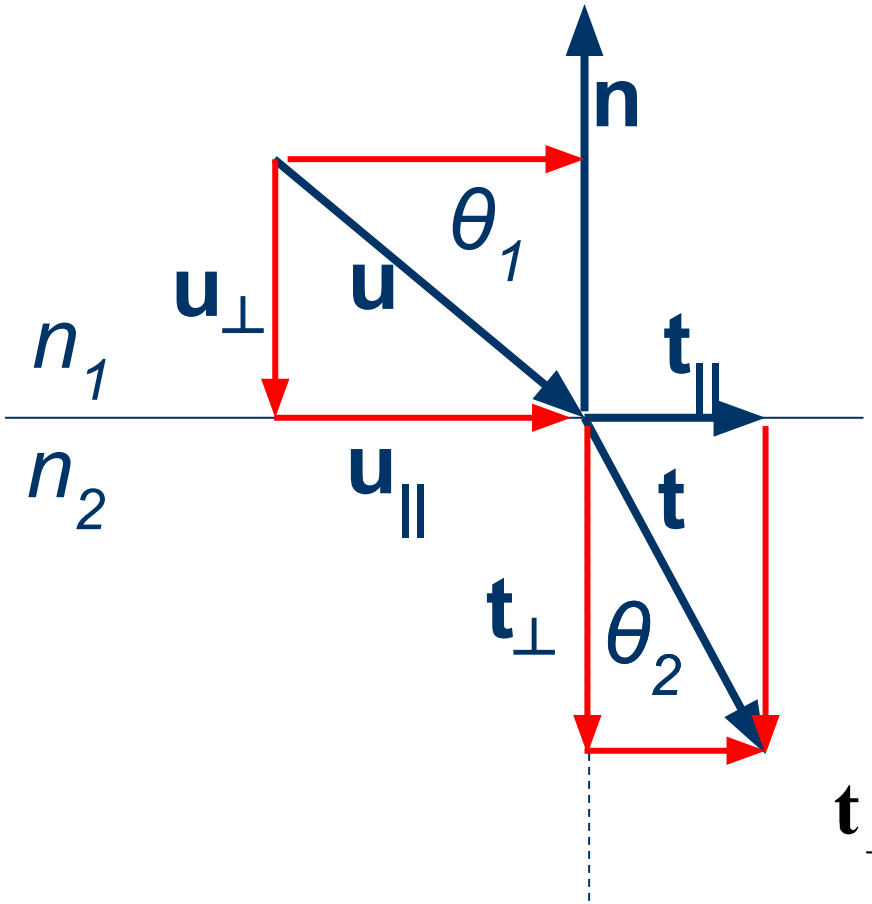
$$\Rightarrow \eta_1 |\mathbf{u}_{\parallel}| = \eta_2 |\alpha \mathbf{u}_{\parallel}|$$

$$\Rightarrow \alpha = \frac{\eta_1}{\eta_2}$$

$$\mathbf{t}_{\parallel} = \frac{\eta_1}{\eta_2}\mathbf{u}_{\parallel}$$

$$= \frac{\eta_1}{\eta_2}\left(\mathbf{u} - (\mathbf{u} \bullet \mathbf{n})\mathbf{n}\right) = \mathbf{z}$$

Transmitted Ray



$$|\mathbf{t}|^2 = |\mathbf{t}_\perp|^2 + |\mathbf{t}_\parallel|^2$$

$$\Rightarrow |\mathbf{t}_\perp| = \sqrt{1 - |\mathbf{t}_\parallel|^2}$$

$$\Rightarrow |-\beta\,\mathbf{n}| = \sqrt{1 - |\mathbf{t}_\parallel|^2}$$

$$\Rightarrow \beta\,|\mathbf{n}| = \sqrt{1 - |\mathbf{t}_\parallel|^2}, \text{ since } |\mathbf{V}| = |-\mathbf{V}|$$

$$\Rightarrow \beta = \sqrt{1 - |\mathbf{t}_\parallel|^2}$$

$$\mathbf{t}_\perp = -\beta\,\mathbf{n} = -\left(\sqrt{1 - |\mathbf{t}_\parallel|^2}\right)\mathbf{n}$$

$$\mathbf{t} = \mathbf{t}_\parallel + \mathbf{t}_\perp$$

$$\mathbf{t} = \mathbf{t}_\parallel - \left(\sqrt{1 - |\mathbf{t}_\parallel|^2}\right)\mathbf{n}$$

Transmitted Ray – Hearn n Baker book

Something might be wrong

Checked, seems fine

$$T = \frac{\eta_i}{\eta_r} u - \left(\cos\theta_r - \frac{\eta_i}{\eta_r} \cos\theta_i \right) N$$

$$\cos\theta_r = \sqrt{1 - \left(\frac{\eta_i}{\eta_r} \right)^2 \left(1 - \cos^2 \theta_i \right)}$$

Transmitted Ray – Peter Shirley book

$$n \sin\theta = n_t \sin\phi. \qquad \cos^2\phi = 1 - \frac{n^2\left(1-\cos^2\theta\right)}{n_t^2}.$$

$$\mathbf{t} = \sin\phi\mathbf{b} - \cos\phi\mathbf{n}.$$

Since we can describe $\mathbf{d}$ in the same basis, and $\mathbf{d}$ is known, we can solve for $\mathbf{b}$:

$$\mathbf{d} = \sin\theta\mathbf{b} - \cos\theta\mathbf{n},$$
$$\mathbf{b} = \frac{\mathbf{d} + \mathbf{n}\cos\theta}{\sin\theta}.$$

This means that we can solve for $\mathbf{t}$ with known variables:

$$\mathbf{t} = \frac{n\left(\mathbf{d} + \mathbf{n}\cos\theta\right)}{n_t} - \mathbf{n}\cos\phi$$
$$= \frac{n\left(\mathbf{d} - \mathbf{n}(\mathbf{d}\cdot\mathbf{n})\right)}{n_t} - \mathbf{n}\sqrt{1 - \frac{n^2\left(1-(\mathbf{d}\cdot\mathbf{n})^2\right)}{n_t^2}}.$$

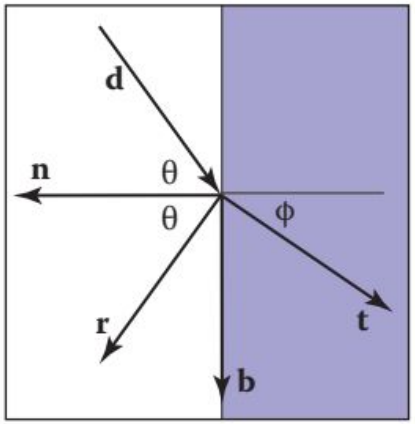


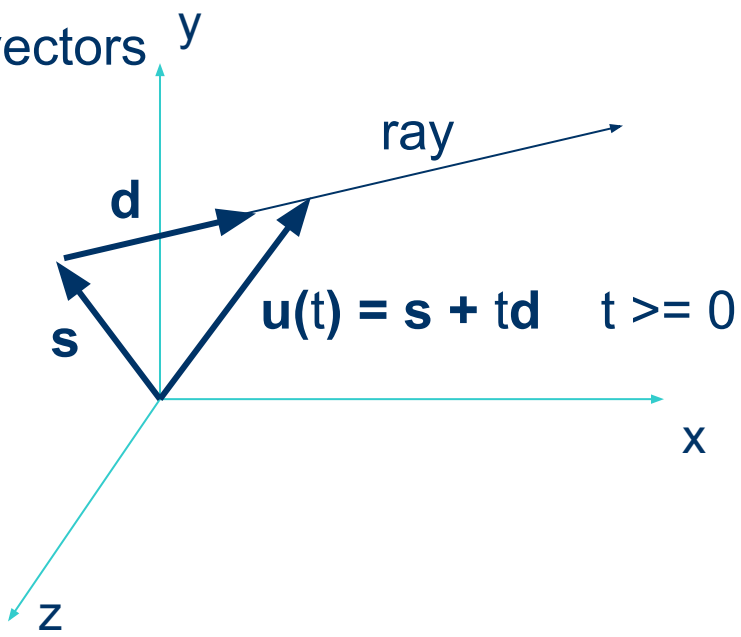
Figure 13.2. The vectors **n** and **b** form a 2D orthonormal basis that is parallel to the transmission vector **t**.

$$P = P_0 + su$$

$$u = \frac{P_{pix} - P_{prp}}{\left| P_{pix} - P_{prp} \right|}$$

Ray - Object Intersection

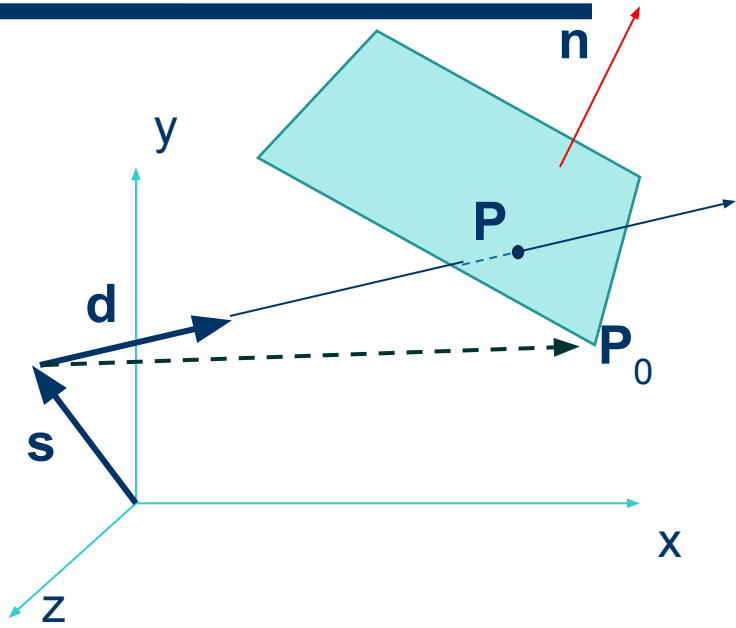
- Representation of ray
  - $\mathbf{s} = [x_s\ y_s\ z_s]^T$ specify the starting point
  - $\mathbf{d} = [x_d\ y_d\ z_d]^T$ describe the direction
  - Assume $|d| = 1$
- Parametric form
  - $\mathbf{u}(t) = \mathbf{s} + t\mathbf{d} \quad (0 \le t)$
    - Where $\mathbf{u}(t)$ denotes a family of vectors



Ray - Object Intersection

- **Ray – Surface intersection**
- Surface
  - $\mathbf{n} = x_n\mathbf{I} + y_n\mathbf{J} + z_n\mathbf{K}$
  - $\mathbf{P_0}\ (x_0, y_0, z_0)$
- Ray, $\mathbf{u} = \mathbf{s} + t\mathbf{d}$ $(0 \le t)$
- A point P(x,y,z) on plane

$$x_n(x - x_0) + y_n(y - y_0) + z_n(z - z_0) = 0$$



- For some t the ray intersects the plane at P, so

$$x_n(x_s + tx_d - x_0) + y_n(y_s + ty_d - y_0) + z_n(z_s + tz_d - z_0) = 0$$

$$t = -\frac{x_n(x_s - x_0) + y_n(y_s - y_0) + z_n(z_s - z_0)}{x_n x_d + y_n y_d + z_n z_d}$$

$$= \frac{n.\overline{sP_0}}{n.d}$$

n.d = 0, parallel ray

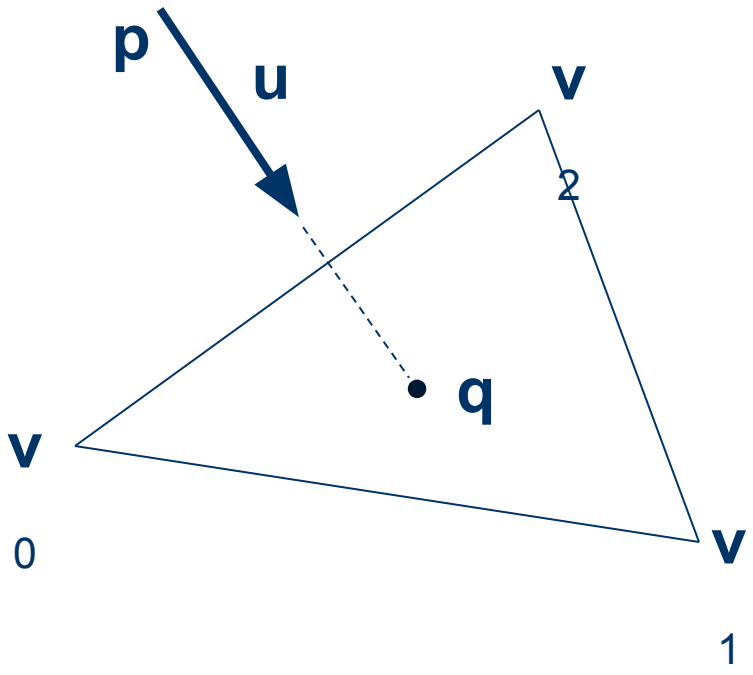
t>0, ray intersects

Ray-Triangle Intersection

- To intersect a ray with a triangle, we must first check if the ray intersects the plane of the triangle
- If we are treating our triangle as one-sided, then we can also verify that the origin of the ray is on the outside of the triangle
- Once we know that the ray hits the plane at point **q**, we must verify that **q** lies inside the 3 edges of the triangle

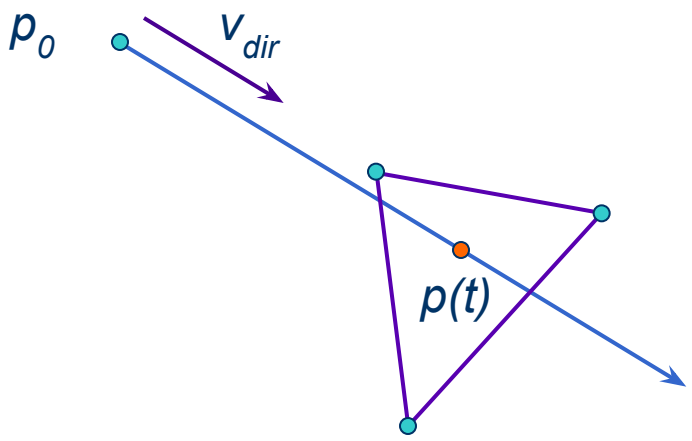
Ray-Triangle

- Does segment **ab** intersect triangle $\mathbf{v}_0\mathbf{v}_1\mathbf{v}_2$?



Ray/Triangle Intersections

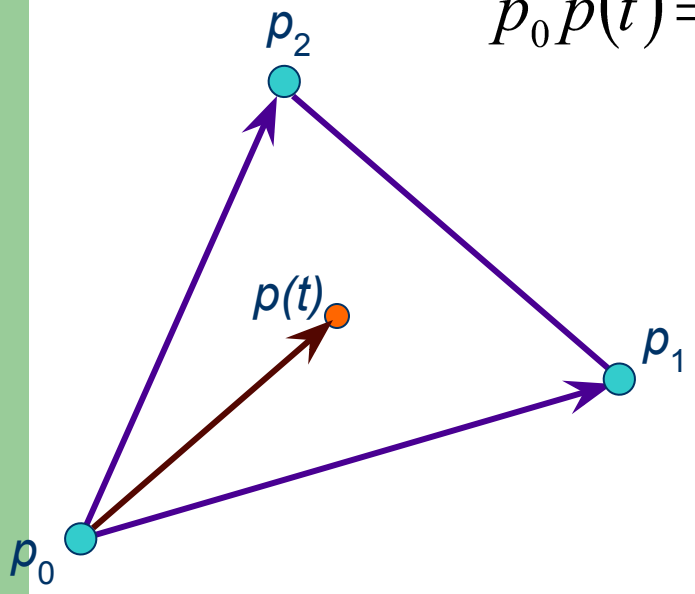
- Several ways to achieve this goal.
  - Badouel's Algorithm.
  - Other Algorithm (close to Segura et al. 98).



Badouel's Algorithm

- First, find the intersection p(t) between the ray and the plane containing the triangle.
- Then use the barycentric coordinates of the point to sort the intersection out (2D algorithm).

$$\overrightarrow{p_0 p(t)} = \alpha . \overrightarrow{p_0 p_1} + \beta . \overrightarrow{p_0 p_2} \quad \alpha > 0, \beta > 0, \alpha + \beta < 1$$



$$x_t - x_0 = \alpha(x_1 - x_0) + \beta(x_2 - x_0)$$
$$y_t - y_0 = \alpha(y_1 - y_0) + \beta(y_2 - y_0)$$
$$z_t - z_0 = \alpha(z_1 - z_0) + \beta(z_2 - z_0)$$

https://ton.github.io/lightbox/2015/04/27/didier-badouel-ray-triangle-intersection.html
https://graphics.stanford.edu/courses/cs348b-98/gg/intersect.html

Badouel's Algorithm

$$\overrightarrow{p_0 p(t)} = \alpha . \overrightarrow{p_0 p_1} + \beta . \overrightarrow{p_0 p_2} \qquad \alpha > 0, \beta > 0, \alpha + \beta < 1$$

$$\vec{a} = \alpha \vec{b} + \beta \vec{c}$$

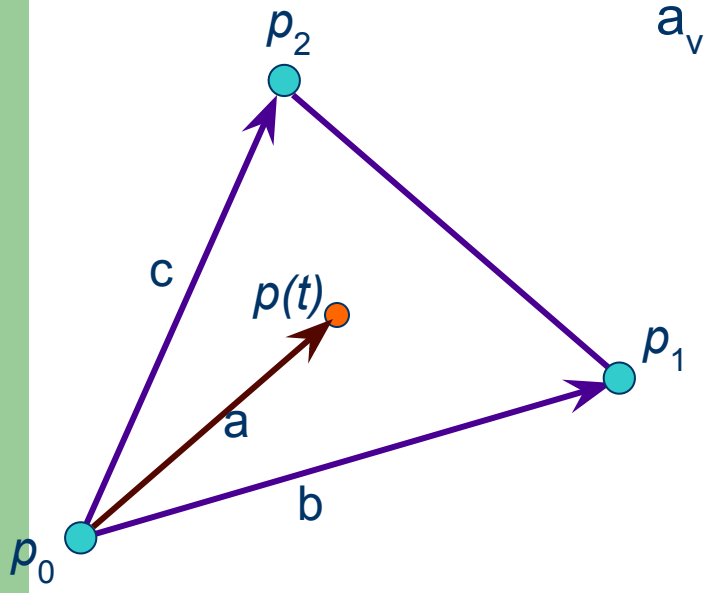
Reduce to 2D: remove smallest dimension say, z

$$x_t - x_0 = \alpha(x_1 - x_0) + \beta(x_2 - x_0)$$

a_u b_u c_u

$$y_t - y_0 = \alpha(y_1 - y_0) + \beta(y_2 - y_0)$$

a_v b_v c_v



$$\begin{pmatrix} b_u & c_u \\ b_v & c_v \end{pmatrix} \begin{pmatrix} \alpha \\ \beta \end{pmatrix} = \begin{pmatrix} a_u \\ a_v \end{pmatrix}$$

$$\alpha = \frac{\begin{vmatrix} a_u & c_u \\ a_v & c_v \end{vmatrix}}{\begin{vmatrix} b_u & c_u \\ b_v & c_v \end{vmatrix}} \qquad \beta = \frac{\begin{vmatrix} b_u & a_u \\ b_v & a_v \end{vmatrix}}{\begin{vmatrix} b_u & c_u \\ b_v & c_v \end{vmatrix}}$$

https://ton.github.io/lightbox/2015/04/27/didier-badouel-ray-triangle-intersection.html

https://graphics.stanford.edu/courses/cs348b-98/gg/intersect.html

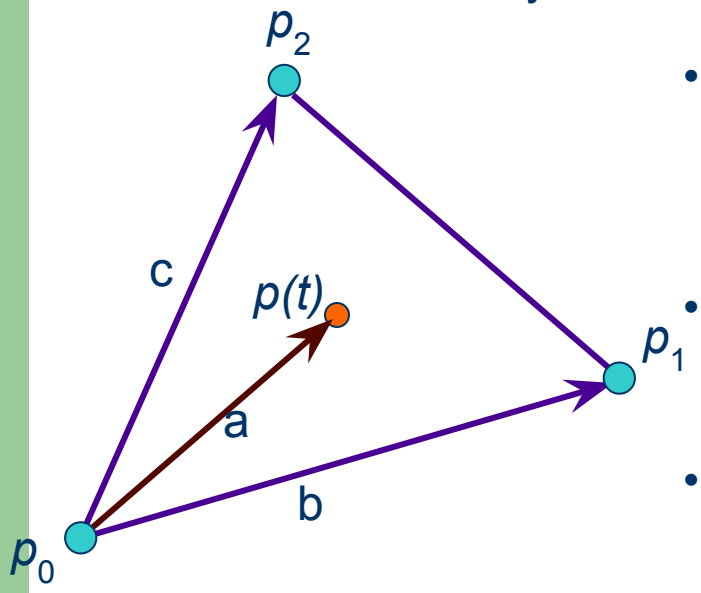
Badouel's Algorithm

$$\overrightarrow{p_0 p(t)} = \alpha . \overrightarrow{p_0 p_1} + \beta . \overrightarrow{p_0 p_2} \qquad \alpha > 0, \beta > 0, \alpha + \beta < 1$$

$$\vec{a} = \alpha \vec{b} + \beta \vec{c}$$

Reduce to 2D: remove smallest dimension say, z

- Badouel now simplifies this system of equations by projecting the triangle on one of the primary planes xy, xz, or yz.
- Badouel uses the fact that after projecting a triangle, the Barycentric coordinate (α,β) of P does not change.



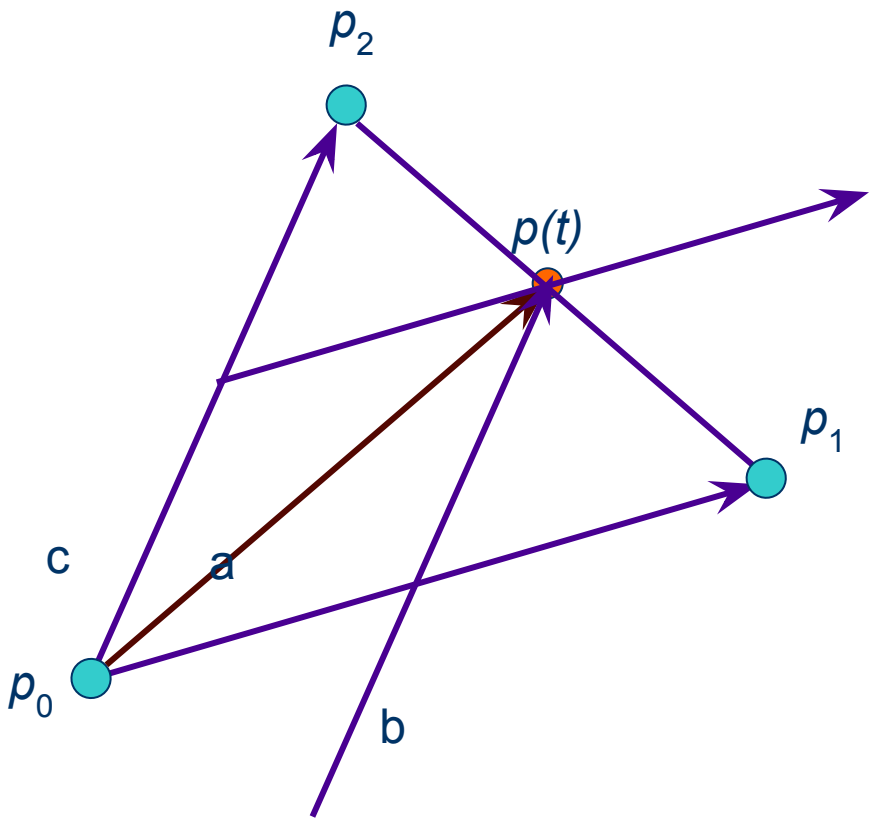
- A requirement for this is that the projected triangle does not degenerate to a line. Thus, we need to make sure that we do not project the triangle on a plane perpendicular to the triangle.
- To avoid this situation, the **dominant** axis of the triangle's normal is determined, and the triangle is **projected** on the plane perpendicular to that axis.
- Note that by taking the dominant axis, the area of the projected triangle is as large as possible, avoiding numerical errors.
- Now, assuming that the X-axis is the dominant axis of the triangle, projecting on the yz plane

Badouel's Algorithm

$$\overrightarrow{p_0 p(t)} = \alpha . \overrightarrow{p_0 p_1} + \beta . \overrightarrow{p_0 p_2} \qquad \alpha > 0, \beta > 0, \alpha + \beta < 1$$

$$\vec{a} = \alpha \vec{b} + \beta \vec{c}$$

Reduce to 2D: remove smallest dimension say, z



Barycentric Coordinates

- Reduce to 2D: remove smallest dimension say, z
- Compute barycentric coordinates

  $a = \mathbf{q} - \mathbf{v}_0$

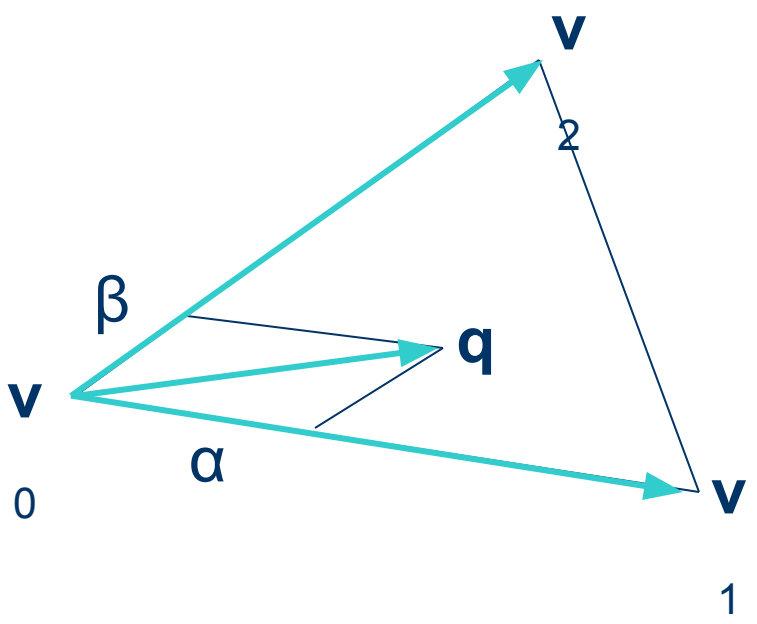
  $\mathbf{b} = \mathbf{v}_1 - \mathbf{v}_0$

  $\mathbf{c} = \mathbf{v}_2 - \mathbf{v}_0$

  $\alpha = (\mathbf{a}_x \cdot \mathbf{c}_y - \mathbf{a}_y \cdot \mathbf{c}_x)/(\mathbf{b}_x \cdot \mathbf{c}_y - \mathbf{b}_y \cdot \mathbf{c}_x)$

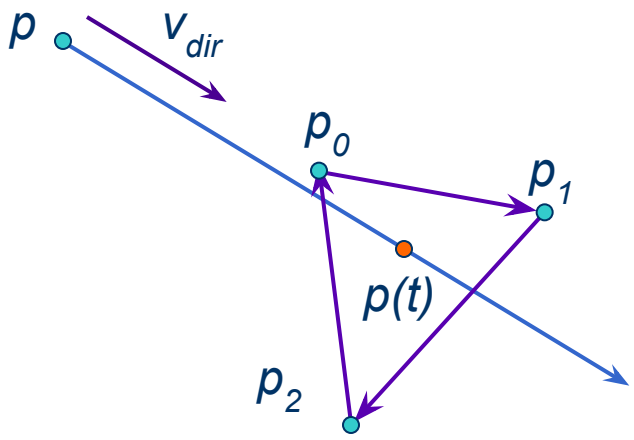
  $\beta = (\mathbf{a}_y \cdot \mathbf{b}_x - \mathbf{a}_x \cdot \mathbf{b}_y)/(\mathbf{b}_x \cdot \mathbf{c}_y - \mathbf{b}_y \cdot \mathbf{c}_x)$

- Reject if $\alpha<0$, $\beta<0$ or $\alpha+\beta>1$



Other Ray/Triangle Algorithms

- The edges of the triangles can be considered as oriented.
- The Ray should always be “on the same side” of the edges
- => Three triple products with the same sign.

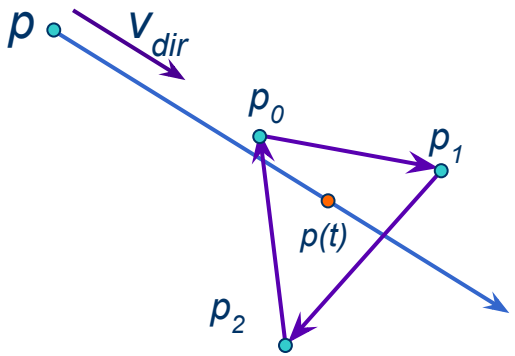


Segura, R.J., Feito, F.R. Segura, R.J., Feito, F.R.: An algorithm for determining intersection segment-polygon in 3D. Computer & Graphics, Vol.22, No.5, pp.587-592, 1998.

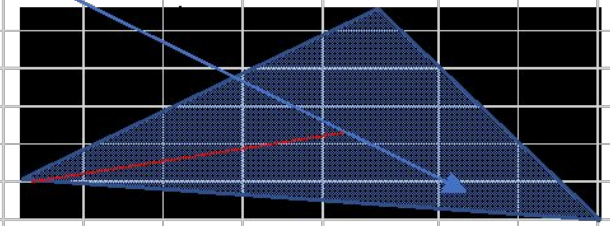
Other Ray/Triangle Algorithms

- There is an intersection Ray/Triangle only if:

$$Sign\left(\left(\overrightarrow{pp_0} \times \overrightarrow{pp_1}\right) \cdot v\right) = Sign\left(\left(\overrightarrow{pp_1} \times \overrightarrow{pp_2}\right) \cdot v\right) = Sign\left(\left(\overrightarrow{pp_2} \times \overrightarrow{pp_0}\right) \cdot v\right)$$



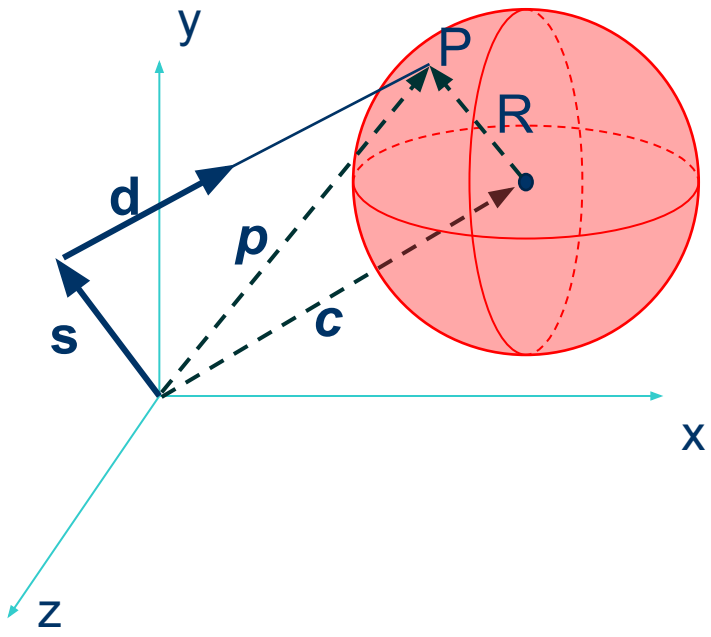
- Fast algorithm.
  - Can take advantage of SIMD instructions on processor.
- But:
  - Only in order to detect an intersection.
  - Extra work must be done in order to get the tri-linear interpolating weights (including a div) and the real intersection location if needed (but not always required).



													x	y	z					
											P0	v0	0	0	0					
												v1	6	-1	-5					
												v2	4	4	1					
												q	1.42	1.21	0.16					
											q-v0	a	1.42	1.21	0.16	1.87	0.76	0.65	0.09	1
											v1-v0	b	6	-1	-5	7.87	0.76	-0.1	-0.6	1
											v2-v0	c	4	4	1	5.74	0.7	0.7	0.17	1
											A		0.84							
	s	1	1	1							B		8.68							
											Det, D		28							
	d	1	0.5	-2		2.2913					beta	A/D	0.03							
		0.44	0.22	-0.9		1					alpha	B/D	0.31							
N	bxc	0.42	-0.6	0.62		0.9434						bu cv	24	l						
		0.45	-0.6	0.66		1						bv cu	-4	m	28	l-m = D				
				dominant is z, project on xy plane																
	n.d	0.19	-0.1	-0.6		-0.5114						au cv	5.68	n						
												av cu	4.84	o	0.84	n-o = E				
	sP0	-1	-1	-1																
												bu av	7.26	r						
	N.sP0	-0.4	0.61	-0.7		-0.4921						bv au	-1.4	s	8.68	r - s = F				
												alpha	0.03							
	t	0.96										beta F	0.31							

Ray - Object Intersection

- Ray – Sphere intersection
  - $\mathbf{c} = x_c\mathbf{I} + y_c\mathbf{J} + z_c\mathbf{K}$
  - $\boldsymbol{p} = x\mathbf{I} + y\mathbf{J} + z\mathbf{K}$
- For sphere
  - $|\mathbf{p} - \mathbf{c}| = R$
- If ray u intersects at P



$$|\mathbf{s} + t\mathbf{d} - \mathbf{c}|^2 = R^2$$

$$|\mathbf{s} + t\mathbf{d} - \mathbf{c}|^2 = (\mathbf{s} - \mathbf{c} + t\mathbf{d}).(\mathbf{s} - \mathbf{c} + t\mathbf{d})$$

$$= |\mathbf{s} - \mathbf{c}|^2 + 2t(\mathbf{s} - \mathbf{c}).\mathbf{d} + t^2 |\mathbf{d}|^2 = R^2$$

$$At^2 + Bt + C = 0$$

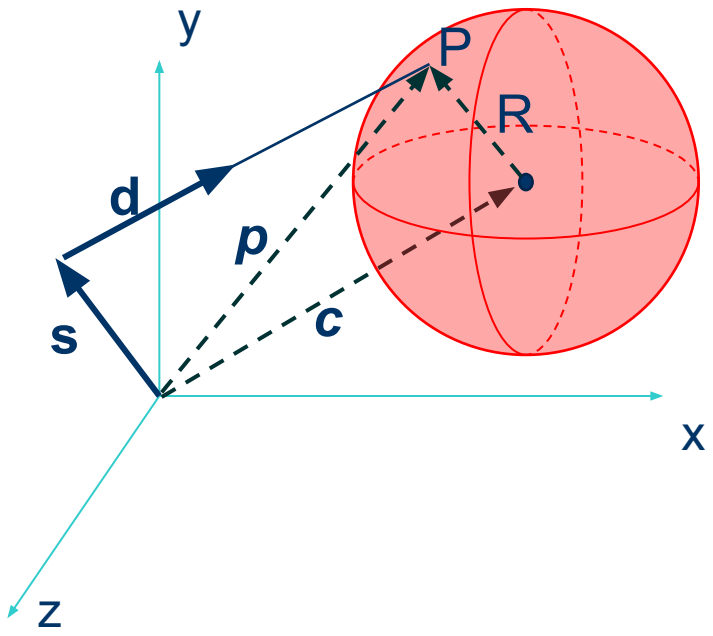
Where, $A = |\mathrm{d}|^2 = 1$; $B = 2(\mathrm{s} - \mathrm{c}).\mathrm{d}$; $C = |\mathrm{s} - \mathrm{c}|^2 - R^2$

Ray - Object Intersection

- **Ray – Sphere intersection**
  - The solution is

$$t = \frac{-B \pm \sqrt{B^2 - 4AC}}{2A}$$

$$= \frac{-B \pm \sqrt{B^2 - 4C}}{2}$$

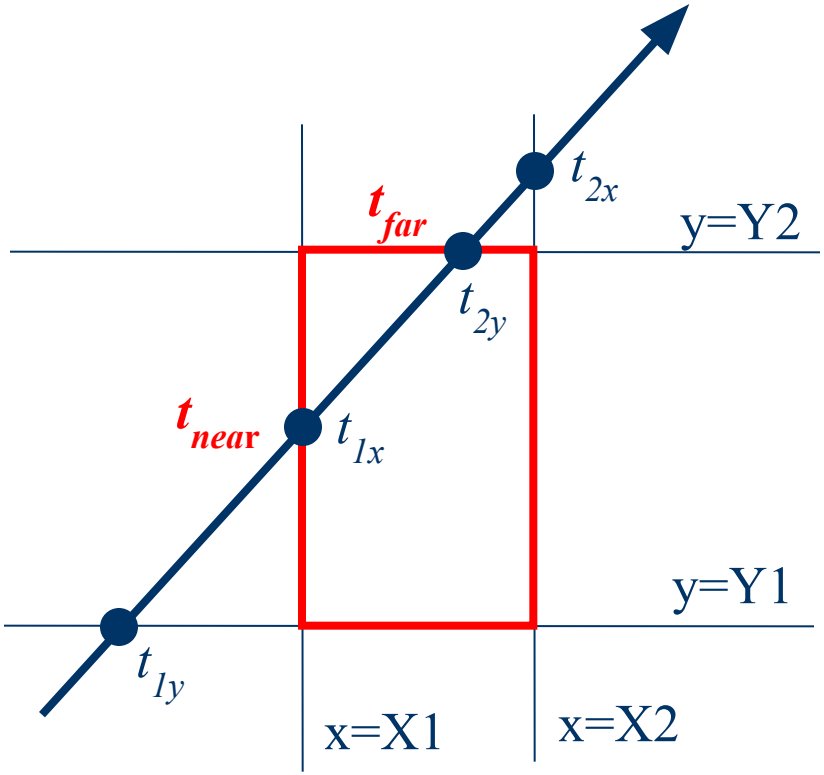


with

$$B^2 - 4C \begin{cases} < 0 & \text{no intersection} \\ = 0 & \text{ray touches sphere} \\ > 0 & \text{two (possible) intersection} \end{cases}$$

Check if t_0, $t_1 > 0$ (ray)
Return $\min(t_0, t_1)$

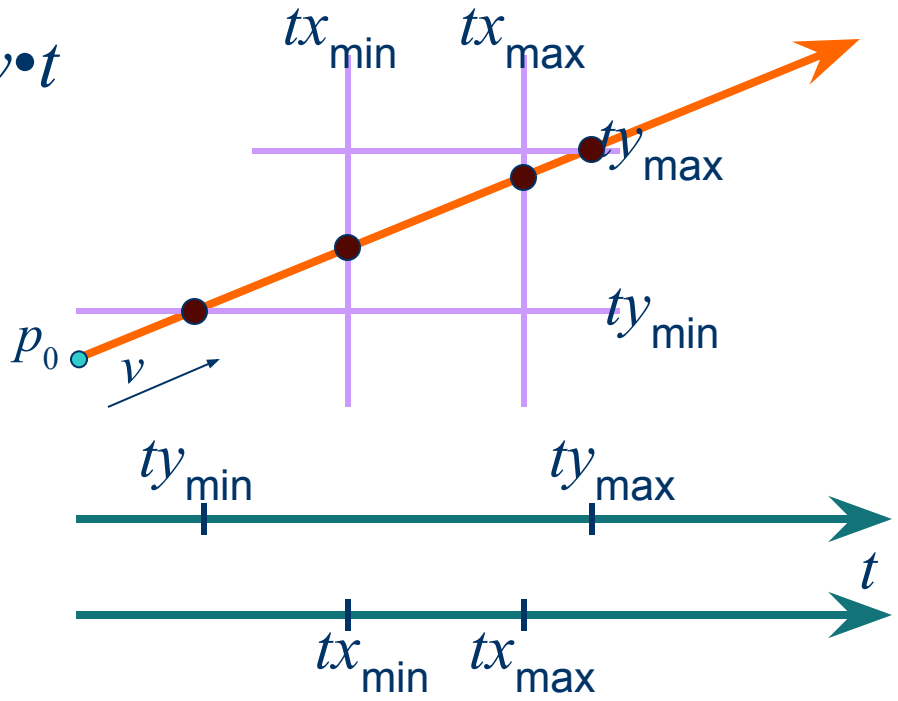
Intersection with Axis-Aligned Box



- For all 3 axes, calculate the intersection distances t_1 and t_2
- $t_{near} = \max(t_{1x}, t_{1y}, t_{1z})$
  $t_{far} = \min(t_{2x}, t_{2y}, t_{2z})$
- If $t_{near} > t_{far}$, box is missed
- If $t_{far} < t_{min}$, box is behind
- If box survived tests, report intersection at t_{near}

Ray/Cube Intersection

$$p(t) = p_0 + v \bullet t$$



The ray intersects the cell only if $\max(tx_{min}, ty_{min}) < \min(tx_{max}, ty_{max})$.

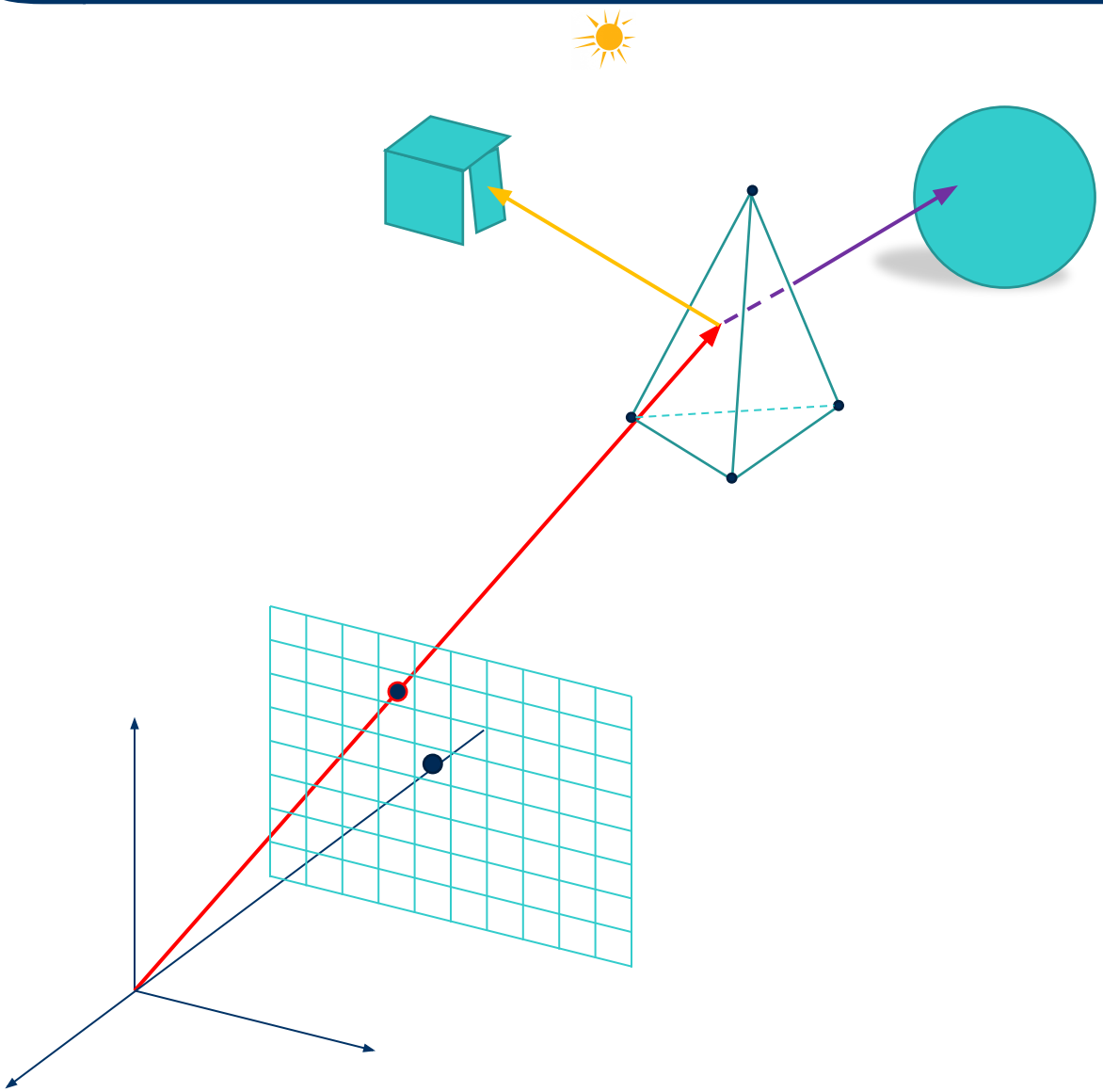
Finding out the exit face consists of finding out the minimum of (tx_{max}, ty_{max}).

Ray/Cube Intersection

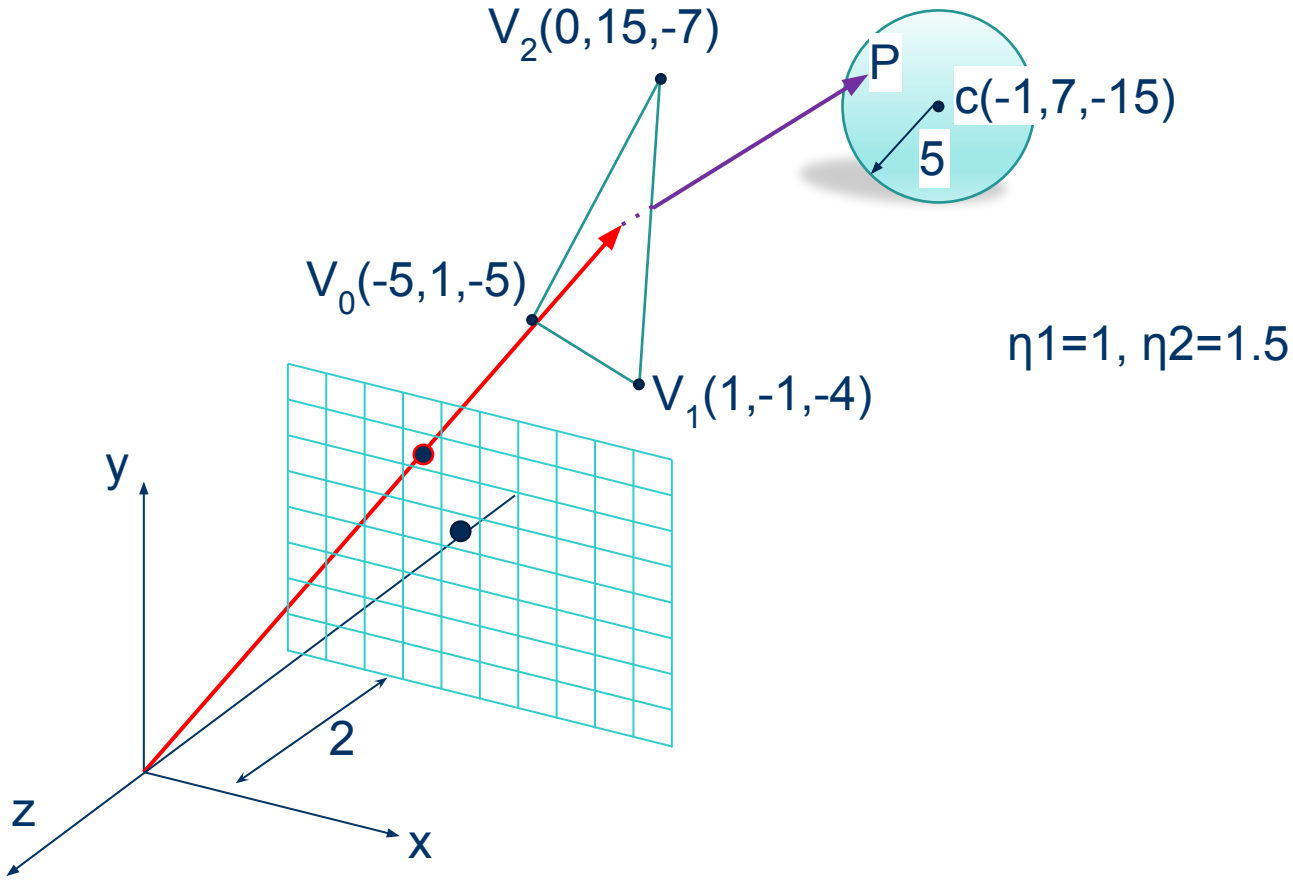
- A cube can be defined with 6 planes (faces).
- 3 pairs of opposite faces (let's reference them as x,y,z) .
- A parametric ray $p(t)$ intersects 2 opposite faces at 2 locations, with 2 respective parameters t_{min} and t_{max}. (let's suppose $t_{min} <= t_{max}$).
- The 6 faces are intersected with 6 different parameters ($t_{xmin}, t_{xmax}, t_{ymin}, t_{ymax}, t_{zmin}, t_{zmax}$).
- The ray intersects the cube only if:

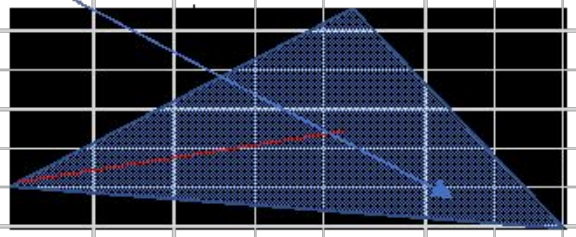
$$\text{Max}(t_{xmin}, t_{ymin}, t_{zmin}) < \text{Min}(t_{xmax}, t_{ymax}, t_{zmax}).$$

Find pixel intensity at (-1,2)

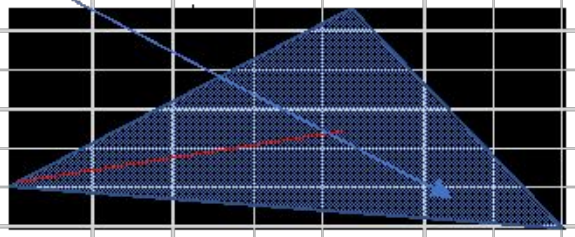


Find P




s
v2
v0
v1

| | | |


s
v2
v0
v1

| | | |

Today

- Motivation – Distribution Ray Tracing
  - Soft shadows
  - Antialiasing (getting rid of jaggies)
  - Glossy reflection
  - Motion blur
  - Depth of field (focus)
- Bounding Boxes
- Spatial Acceleration Data Structures
- Flattening the Transformation Hierarchy

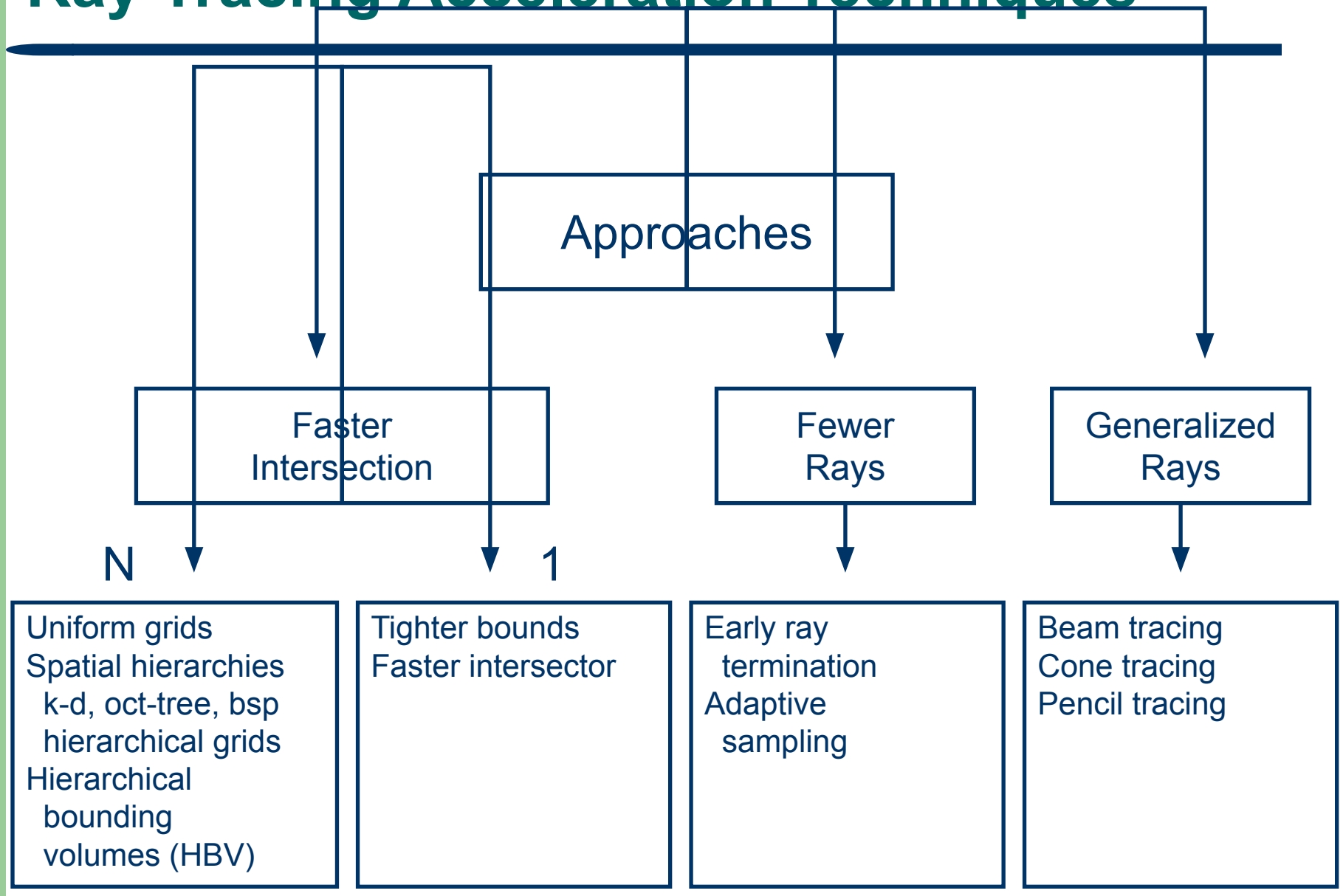
Ray Tracing Algorithm Analysis

- Ray casting
- Lots of primitives
- Recursive
- Distributed Ray Tracing Effects
  - Soft shadows
  - Anti-aliasing
  - Glossy reflection
  - Motion blur
  - Depth of field

cost ≈ height * width *
num primitives *
intersection cost *
size of recursive ray tree *
num shadow rays *
num supersamples *
num glossy rays *
num temporal samples *
num focal samples *
. . .

can we reduce this?

Ray Tracing Acceleration Techniques



Beam Tracing.

- Proposed by:

  Paul S. Heckbert and Pat Hanrahan. BEAM TRACING POLYGONAL OBJECTS. Siggraph 1984, pp. 119-127.

- Takes advantages of spatial coherency between rays to group them and save computations.
- Rays are distributed after Reflections and Refractions according to the reflection/refraction coefficients.
- Constructs a beam tree.
- Outdated.

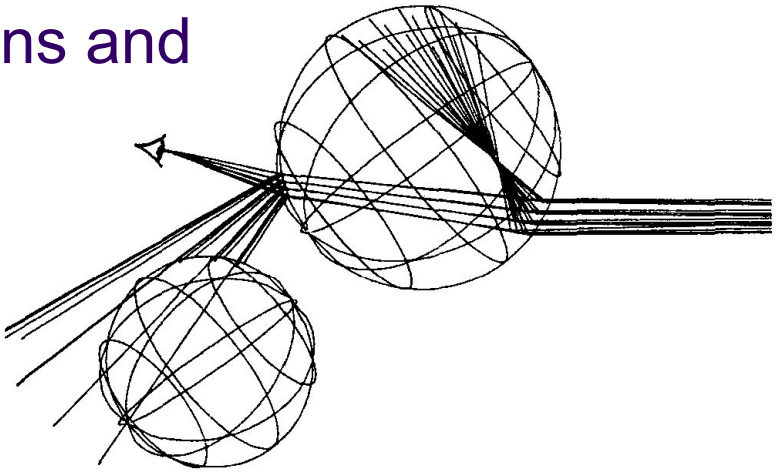


Figure 1. A bundle of rays passing through two spheres.

Pyramidal Ray-Tracing.

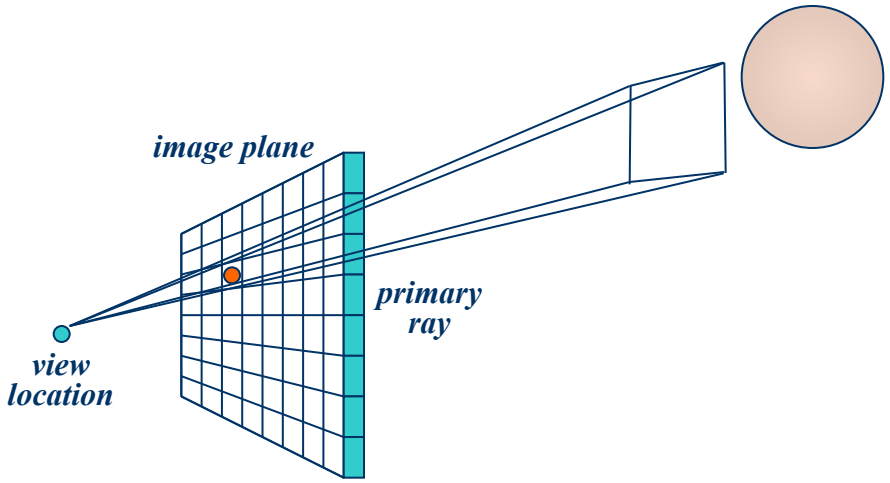
- Proposed by:

  Whitted, Turner. An Improved Illumination Model for Shaded Display. C.A.C.M. vol. 23, no. 6, June 1980, pp. 343-3~19.

- And improved by:

  J. Genetti, D. Gordon and G. Williams. Adaptive supersampling in object space using pyramidal rays. Computer Graphics Forum 17 (1998) 29-54.

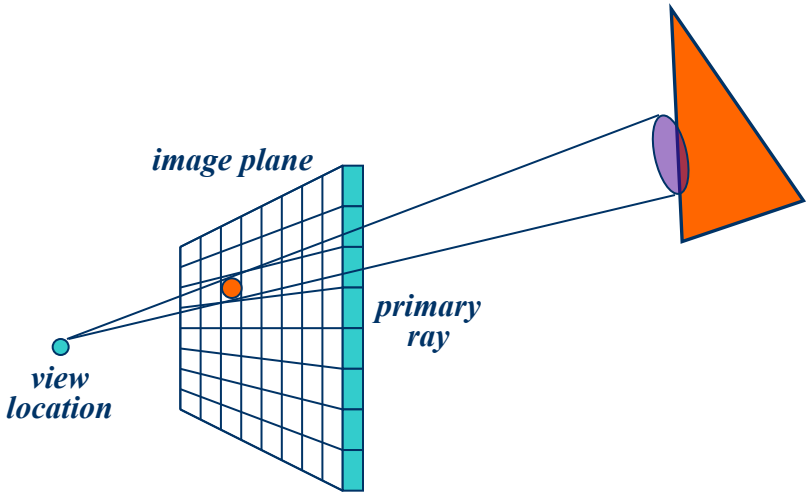
- Main Idea: Replacing Rays by Pyramids to improve aliasing.

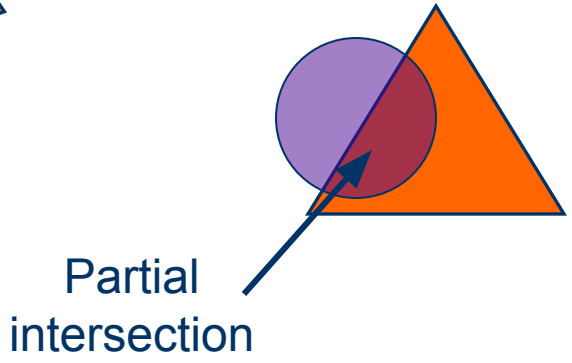


Cone Tracing.

- Proposed by:

  John Amanatides. Ray Tracing with Cones. Siggraph 1984, pp. 129-135.





Acceleration Structures

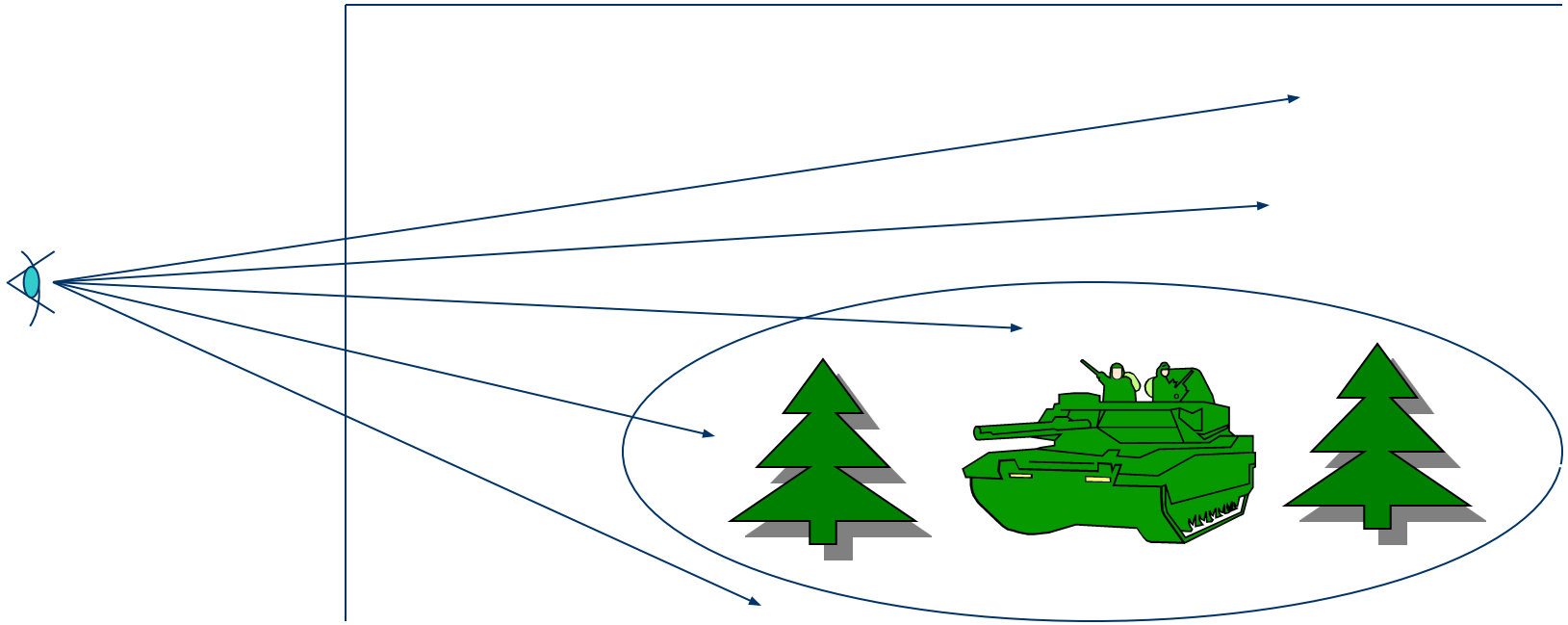
- Complex scenes can contain millions of primitives, and ray tracers need to trace millions of rays
- This means zillions of potential ray-object intersections
- If every ray simply looped through every object and tested if it intersected, we would spend forever just doing loops, not even counting all of the time doing the intersection testing
- Therefore, it is absolutely essential to employ some sort of *acceleration structure* to speed up the ray intersection testing
- An acceleration structure is some sort of data structure that groups objects together into some arrangement that enables the ray intersection to be sped up by limiting which objects are tested
- There are a variety of different acceleration structures in use, but most of the successful ones tend to be based on some variation of hierarchical subdivision of the space around the group of objects

Bounding Volume Hierarchies

- The basic concept of a bounding volume hierarchy is a complex object in a hierarchy of simpler ones
- This works much like the hierarchical
- For example, if one were using spheres as their bounding volume, we could enclose the entire scene in one big sphere
- Within that sphere are several other spheres, each containing more spheres, until we finally get to the bottom level where spheres contain actual geometry like triangles
- To test a ray against the scene, we traverse the hierarchy from the top level
- When a sphere is hit, we test the spheres it contains, and ultimately the triangles/primitives within
- In general, a bounding volume hierarchy can reduce the ray intersection time from O(n) to O(log n), where n is the number of primitives in the scene
- This reduction from linear to logarithmic performance makes a huge difference and makes it possible to construct scenes with millions of primitives

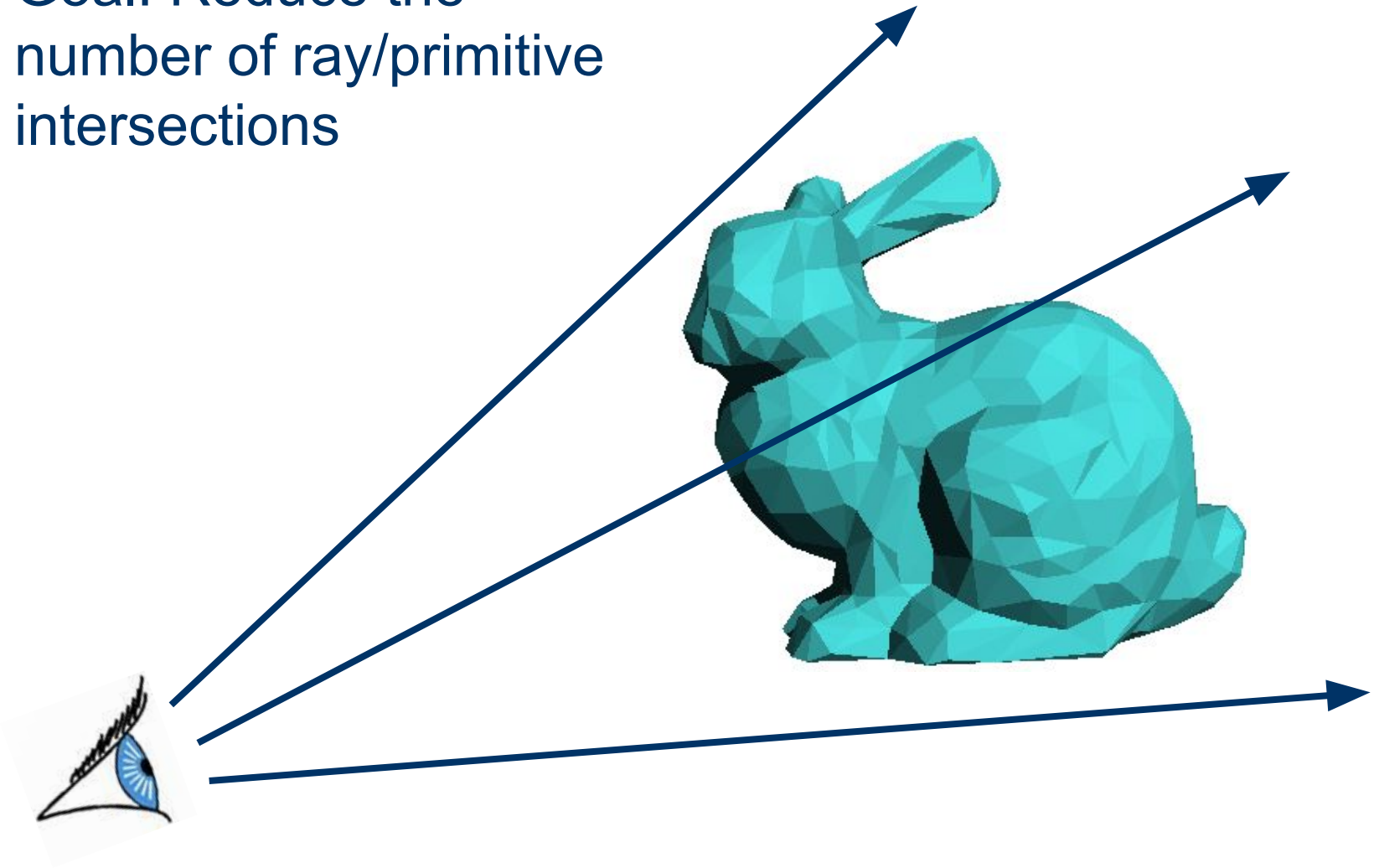
Bounding Volume

Enclosing Groups of objects in sets of bounding volumes and first test for intersection with the bounding volume.



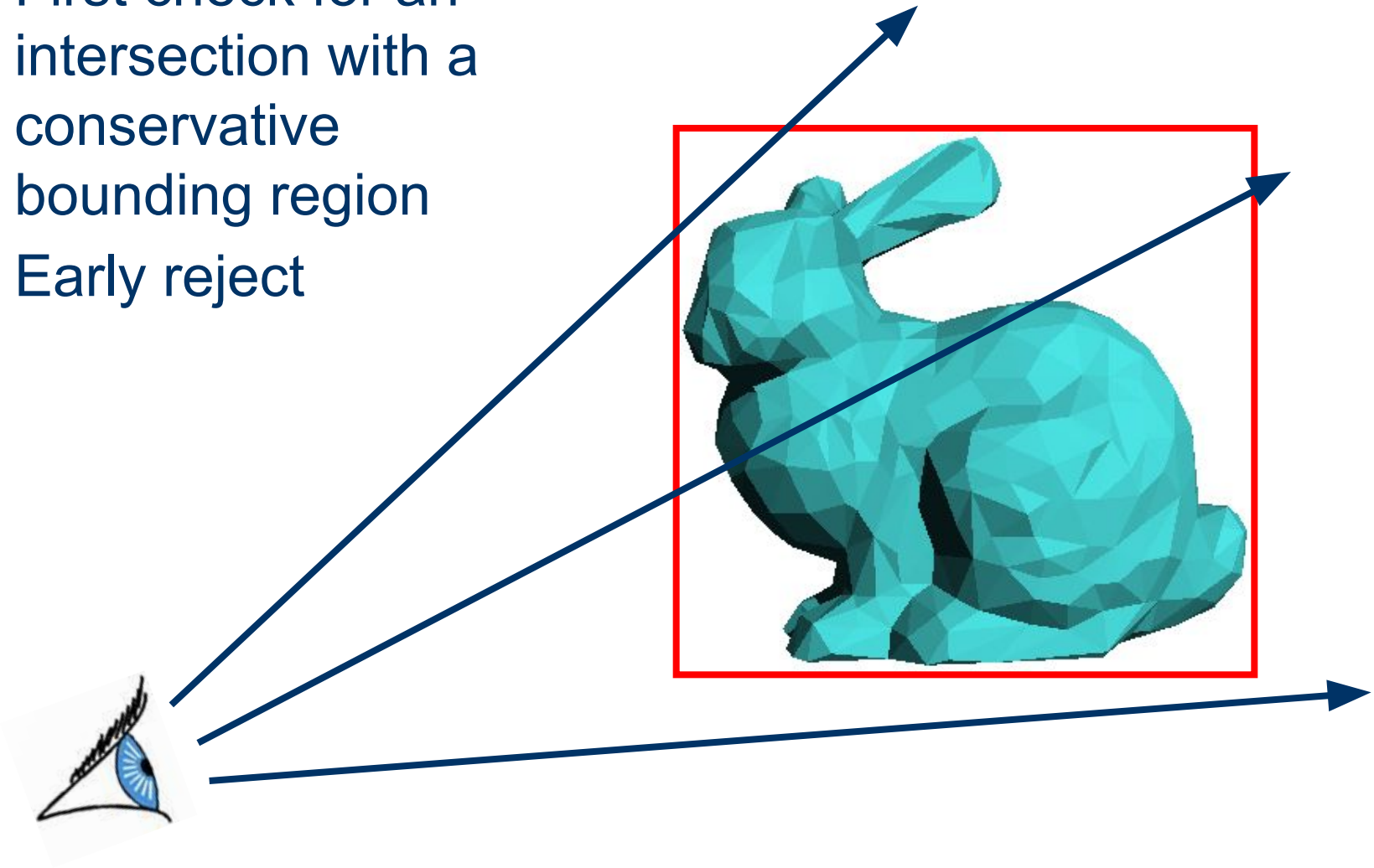
Acceleration of Ray Casting

- Goal: Reduce the number of ray/primitive intersections



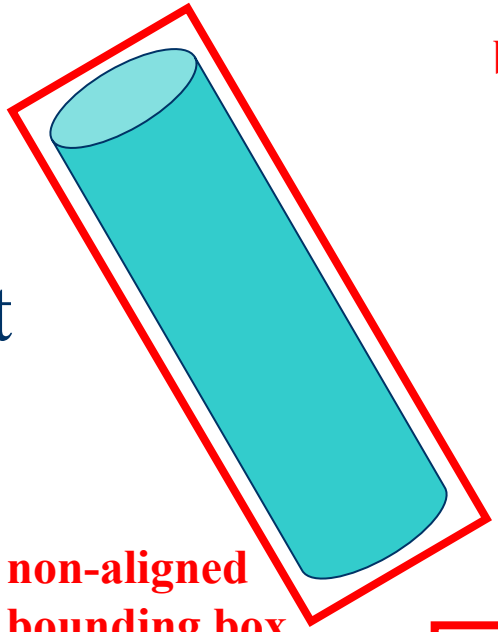
Conservative Bounding Region

- First check for an intersection with a conservative bounding region
- Early reject

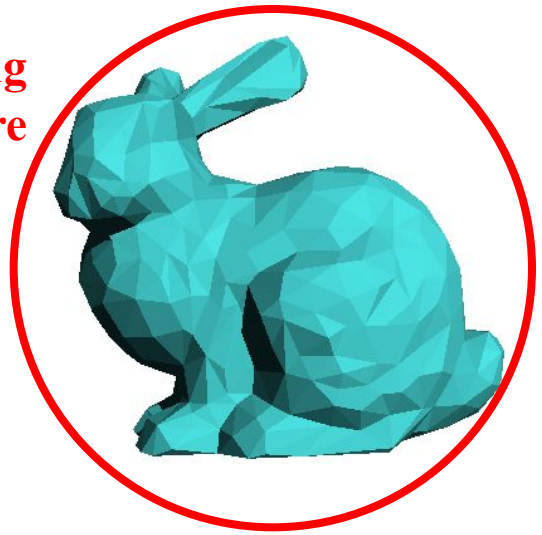


Conservative Bounding Regions

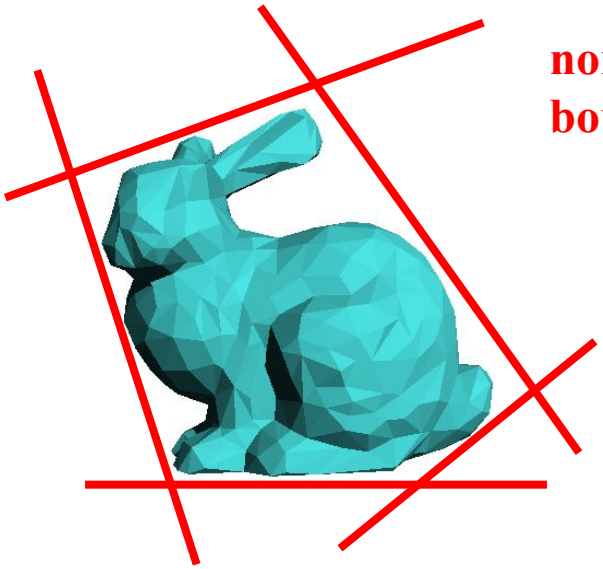
- tight → avoid false positives
- fast to intersect



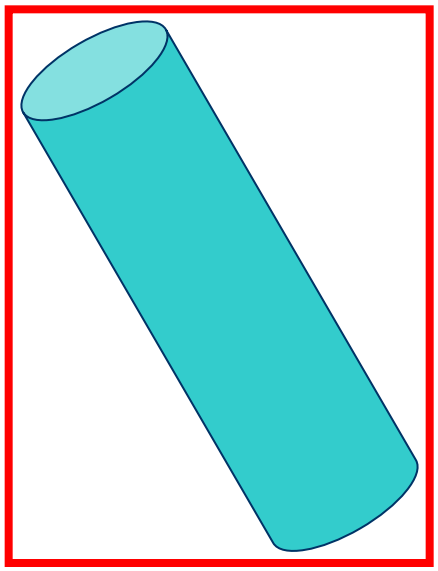
non-aligned bounding box



bounding sphere



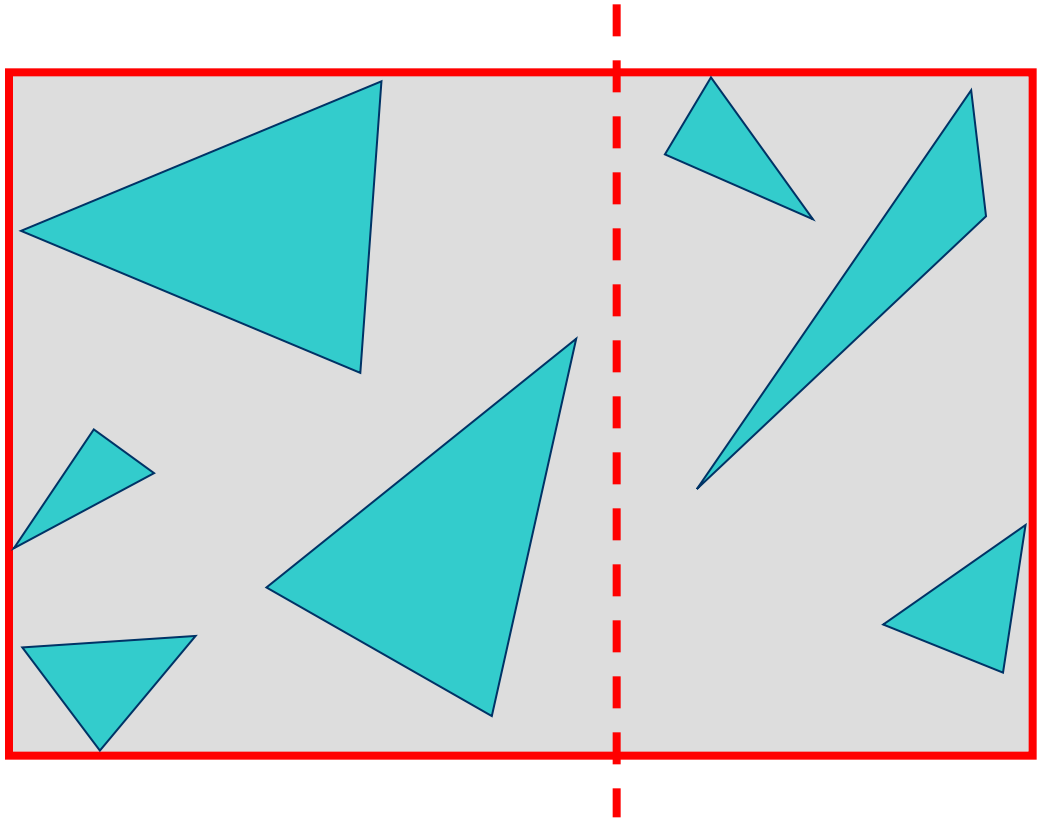
arbitrary convex region (bounding half-spaces)



axis-aligned bounding box

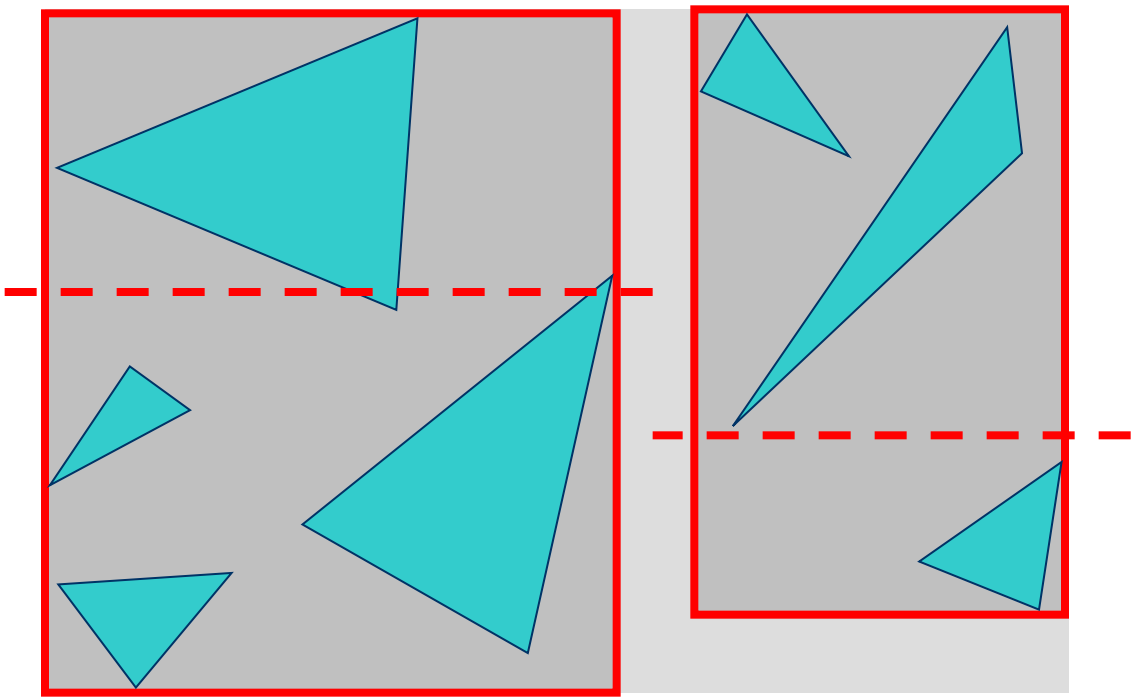
Bounding Volume Hierarchy

- Find bounding box of objects
- Split objects into two groups
- Recurse



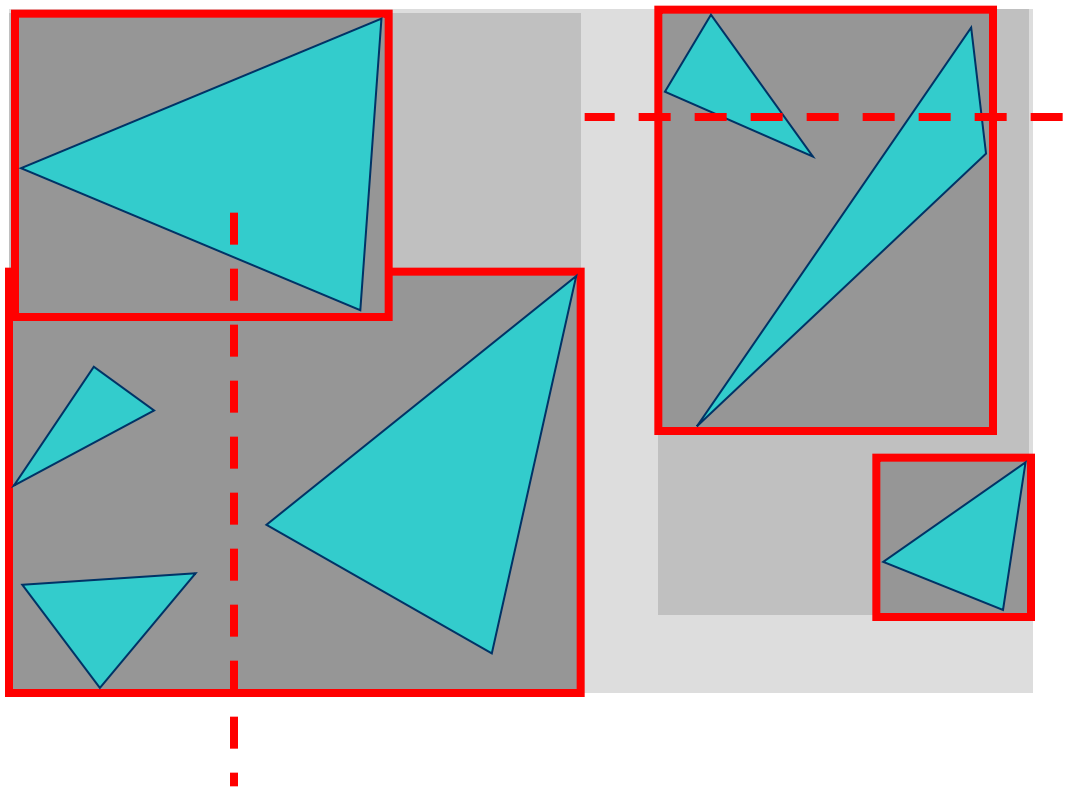
Bounding Volume Hierarchy

- Find bounding box of objects
- Split objects into two groups
- Recurse



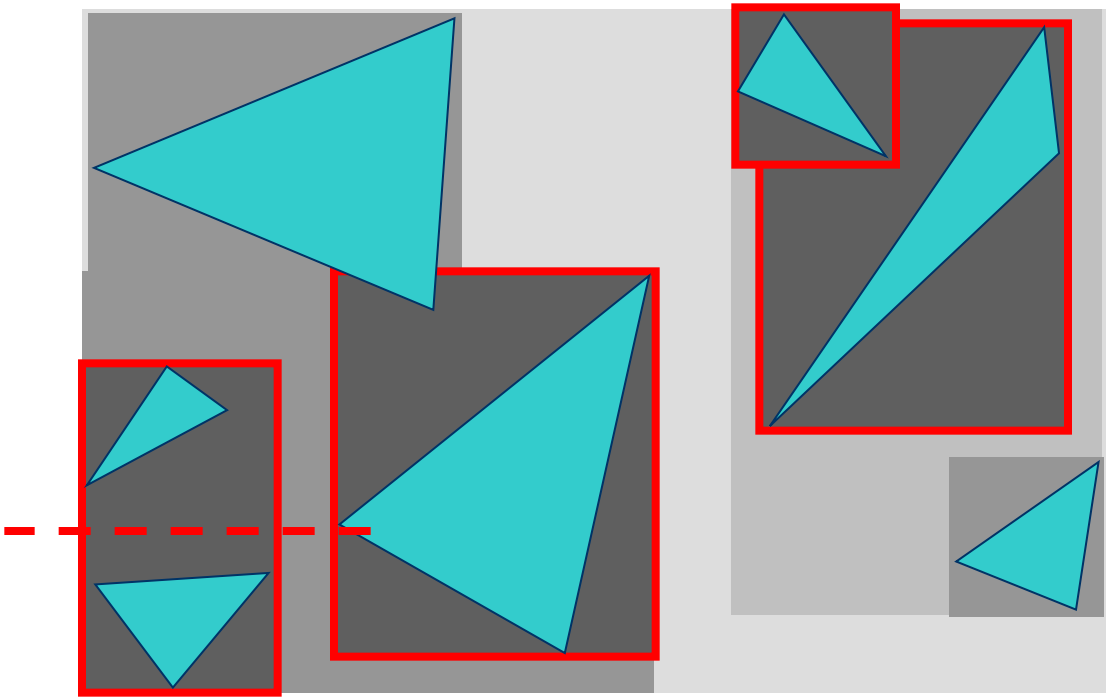
Bounding Volume Hierarchy

- Find bounding box of objects
- Split objects into two groups
- Recurse



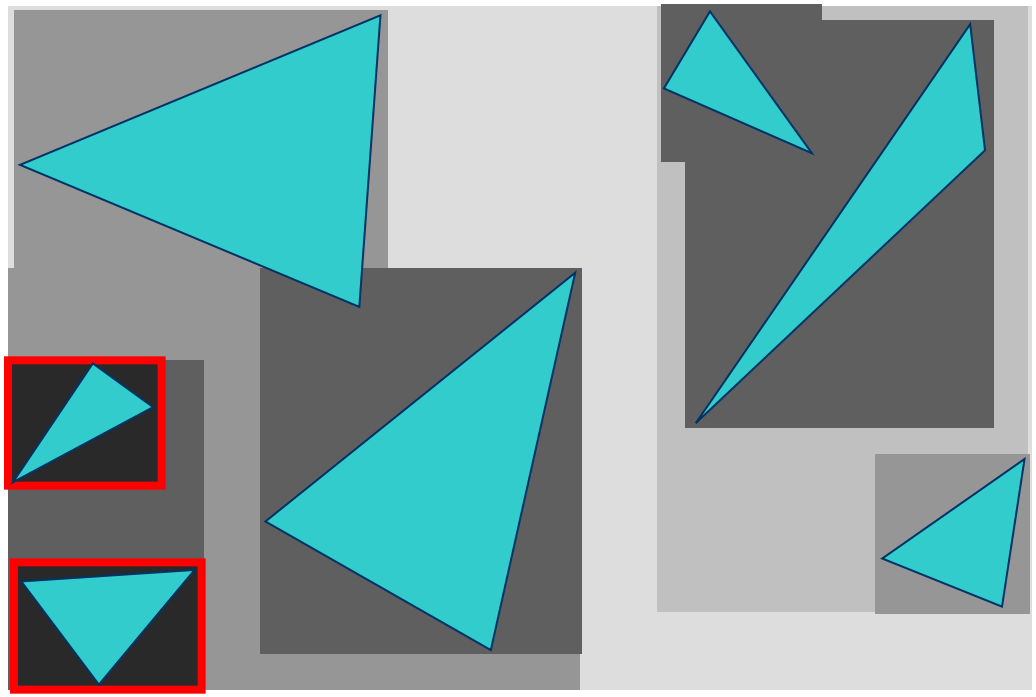
Bounding Volume Hierarchy

- Find bounding box of objects
- Split objects into two groups
- Recurse



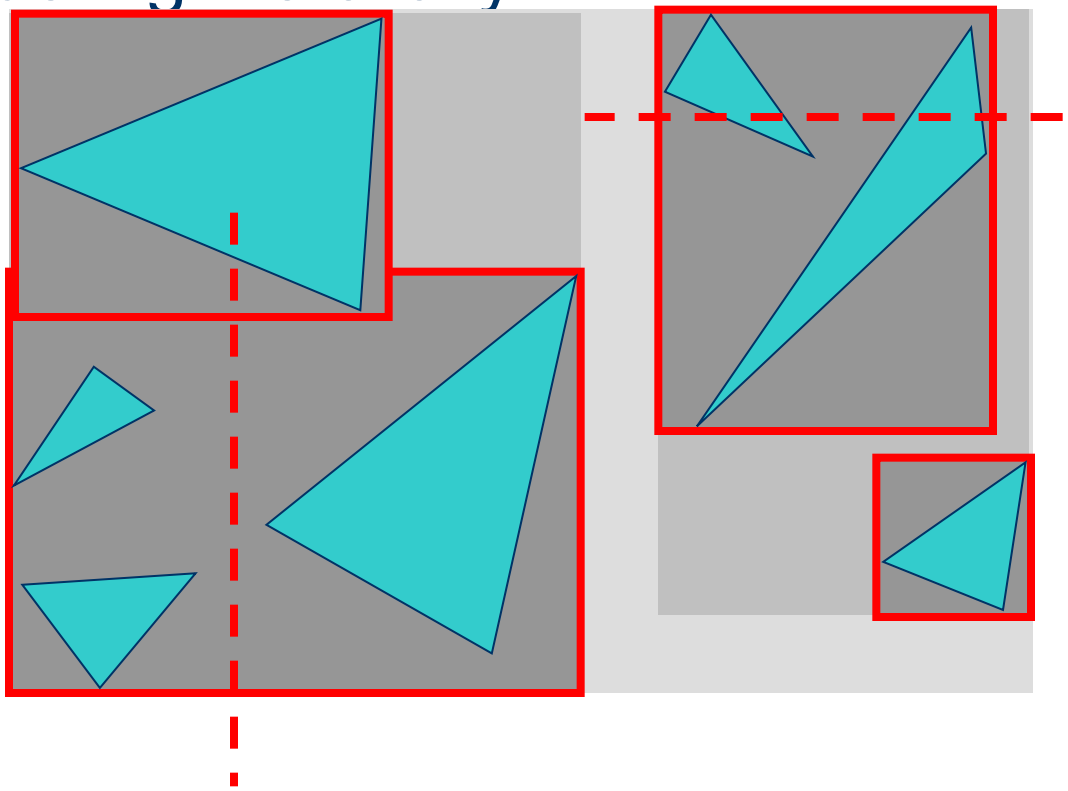
Bounding Volume Hierarchy

- Find bounding box of objects
- Split objects into two groups
- Recurse



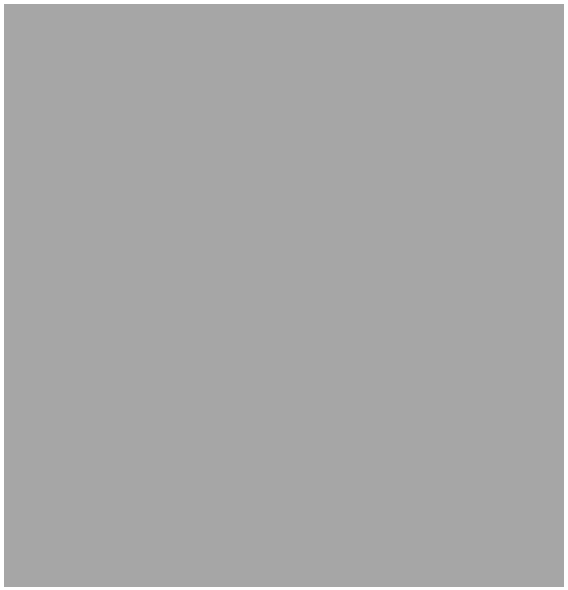
Where to split objects?

- At midpoint *OR*
- Sort, and put half of the objects on each side *OR*
- Use modeling hierarchy



Sphere Hierarchies

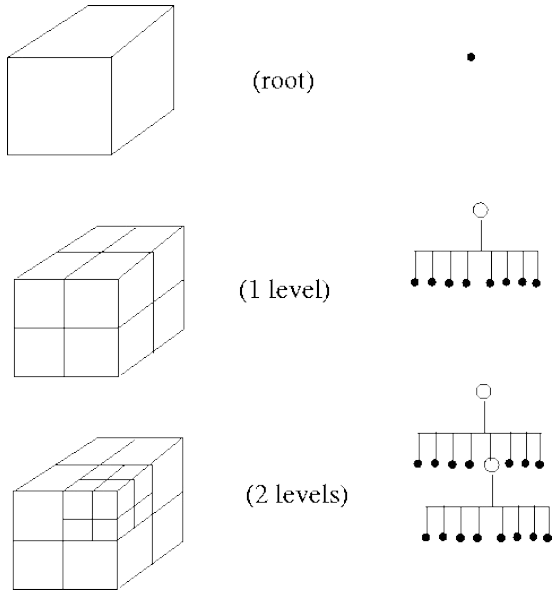
- The sphere hierarchy makes for a good example of the concept, but in practice, sphere hierarchies are not often used for ray tracing
- One reason is that it is not clear how to automatically group an arbitrary set of triangles into some number of spheres, so various heuristic options exist
- Also, as the spheres are likely to overlap a lot, they end up triggering a lot of redundant intersection tests

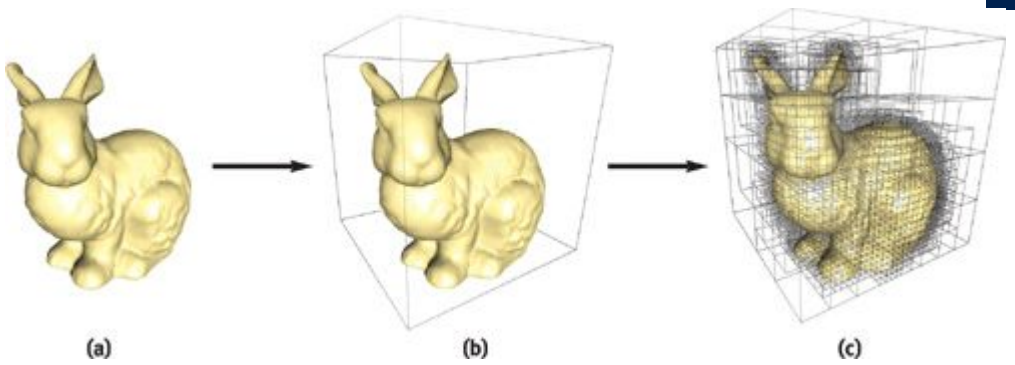


Octrees

- The octree starts by placing a cube around the entire scene
- If the cube contains more than some specified number of primitives (say, 10), then it is split equally into 8 cubes, which are then recursively tested and possibly resplit
- The octree is a more regular structure than the sphere tree and provides a clear rule for subdivision and no overlap between cells
- This makes it a better choice usually, but still not ideal

Octrees

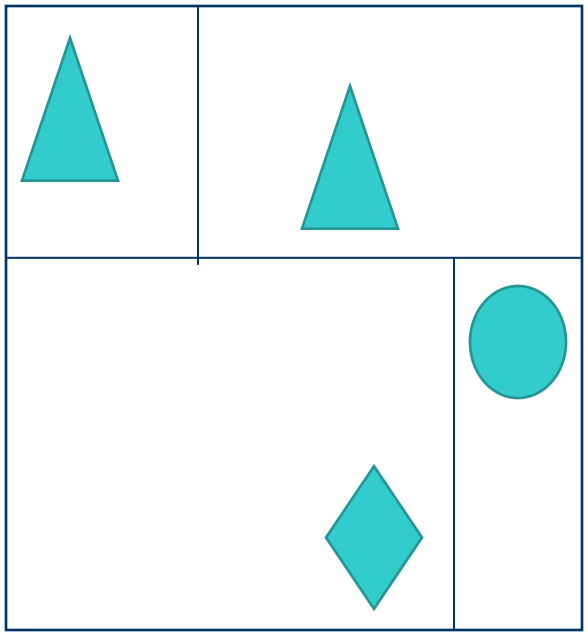




KD Trees

- The KD tree starts by placing a box (not necessarily a cube) around the entire scene
- If the box contains too many primitives, it is split, as with the octree
- However, the KD tree only splits the box into two boxes, that need not be equal
- The split can take place on the x, y, or z place at some arbitrary point within the box
- This makes the KD tree a little bit more adaptable to irregular geometry and able to customize a tighter fit
- In general, KD trees tend to be pretty good for ray tracing
- Their main drawback is that the tree depth can get rather deep, causing the ray intersection to spend a lot of time traversing the tree itself, rather than testing intersections with primitives

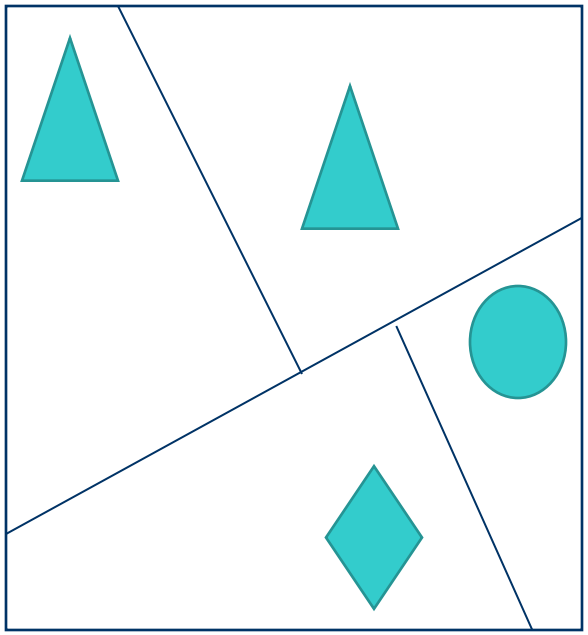
KD Trees



BSP Trees

- The BSP tree (*binary space partitioning*) is much like the KD tree in that it continually splits space into two (not necessarily equal) halves
- Unlike the KD tree which is limited to xyz axis splitting, the BSP tree allows the splitting plane to be placed anywhere in the volume and aligned in any direction
- This makes it a much more difficult problem to choose the location of the splitting plane, and so many heuristics exist
- In practice, BSP trees tend to perform well for ray tracing, much like KD trees

BSP Trees

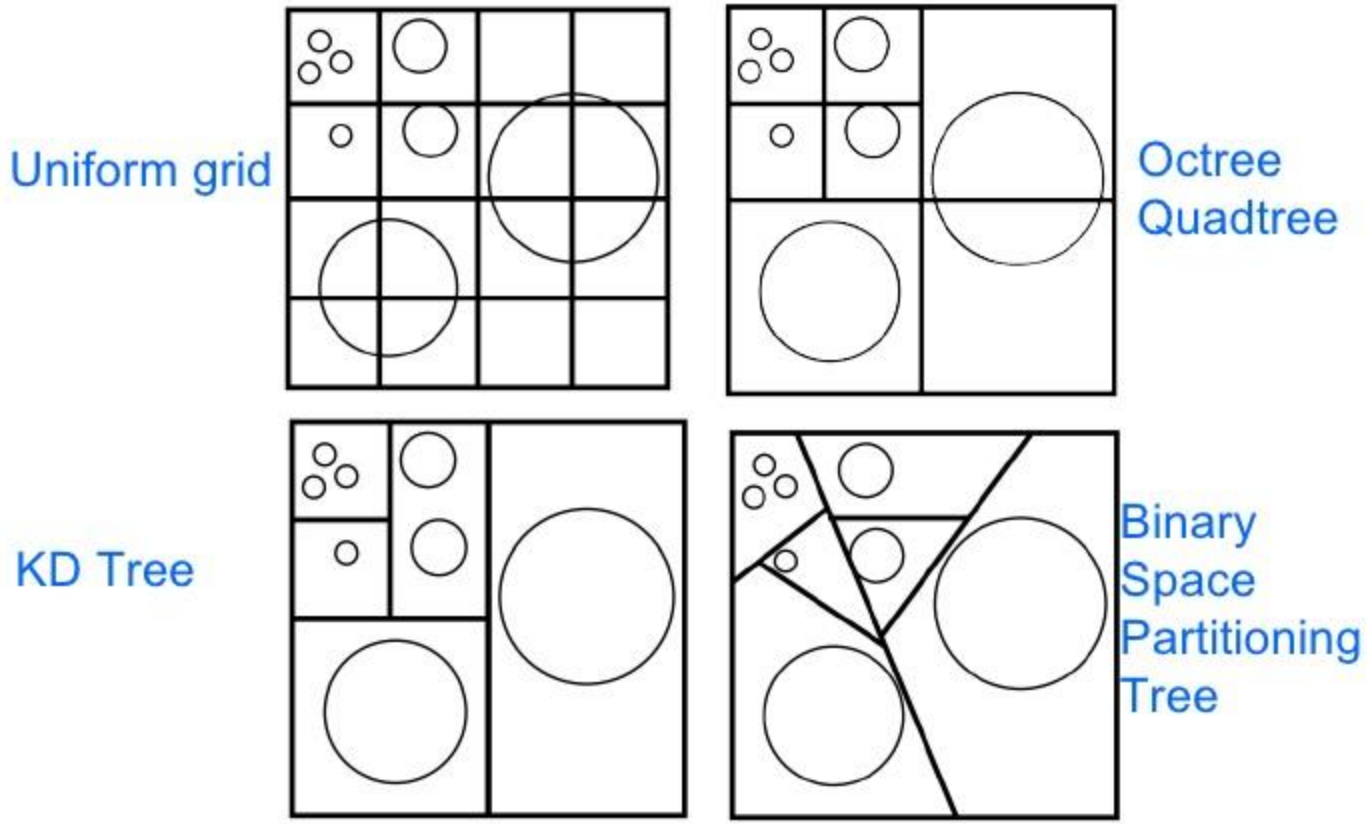


Uniform Grids

- One can also subdivide space into a uniform grid, instead of hierarchically
- This is fast for certain situations, but gets too expensive in terms of memory for large complex scenes
- It also tends to loose its performance advantages in situations where primitives have a large variance in size and location (which is common)
- As a result, they are not really a practical general purpose acceleration structure for ray tracing

Uniform Grids

Space Subdivision Approaches



Distribution Ray Tracing



Distribution Ray Tracing

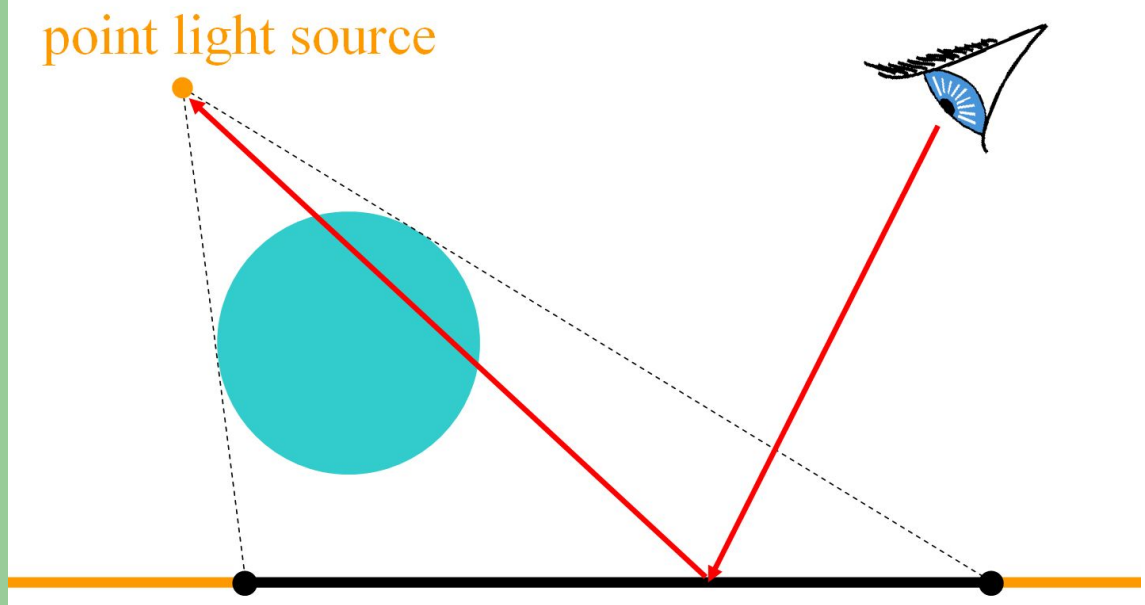
- In 1984, an important modification to the basic ray tracing algorithm was proposed, known as *distributed ray tracing*
- The concept basically involved shooting several distributed rays to achieve what had previously been done with a single ray
- The goal is not to simply make the rendering slower, but to achieve a variety of ‘soft’ lighting effects such as antialiasing, camera focus, soft-edge shadows, blurry reflections, color separation, motion blur, and more
- As the term ‘distributed’ tends to refer to parallel processing in modern days, the distributed ray tracing technique is now called *distribution ray tracing*, and the term ‘distributed’ is reserved for parallel ray tracing, which is also an important subject

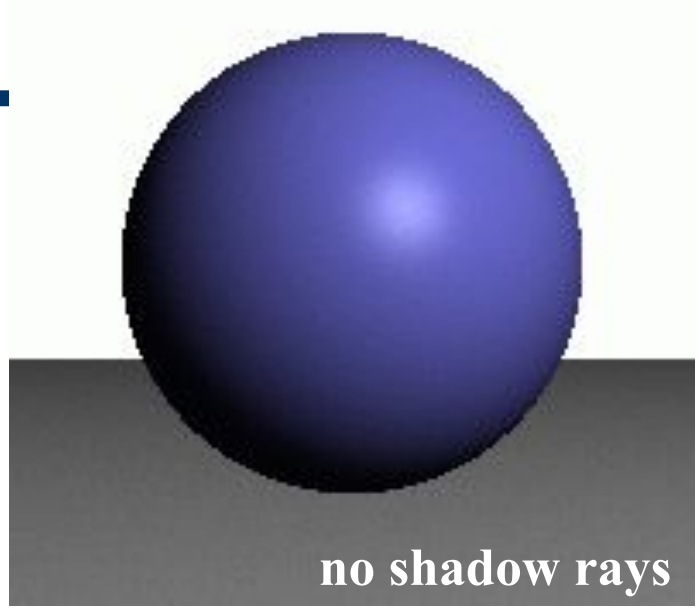
Soft Shadows

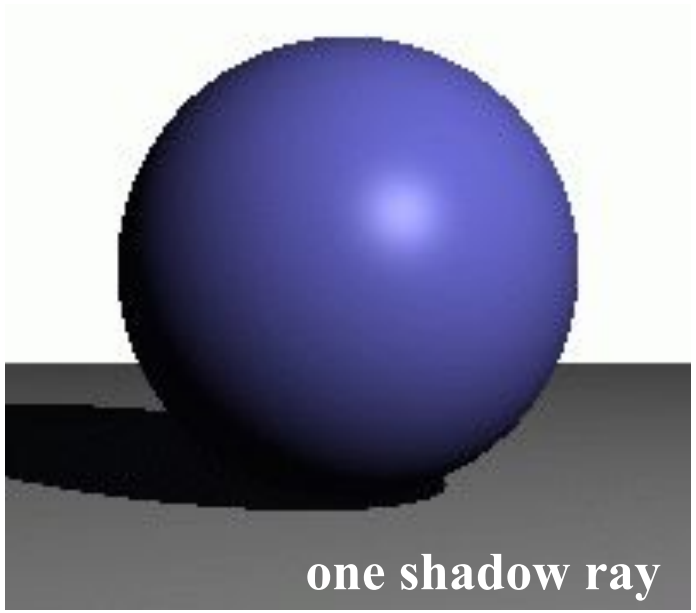
- One nice visual effect we can achieve with distribution ray tracing is *soft shadows*
- Instead of treating a light source as a point and shooting a single ray to test for shadows, we can treat the light source as a sphere and shoot several rays to test for partial blocking of the light source
- If 15% of the shadow rays are blocked, then we get 85% of the incident light from the light source
- In lighting terminology, the completely shadowed region is called the *umbra* and the partially shadowed region is called the *penumbra*

Shadows

- one shadow ray per intersection per point light source







Shadows & Light Sources

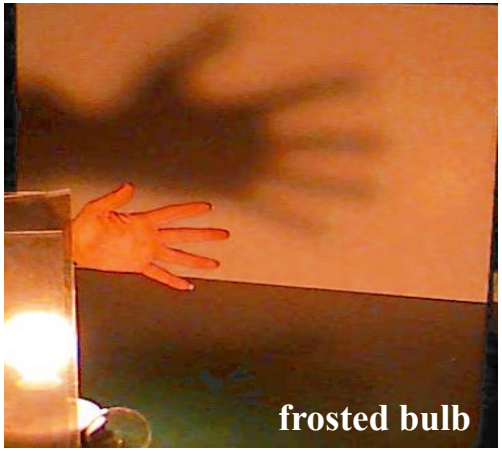


http://3media.initialized.org/photos/2000-10-18/index_gall.htm



http://www.davidfay.com/index.php





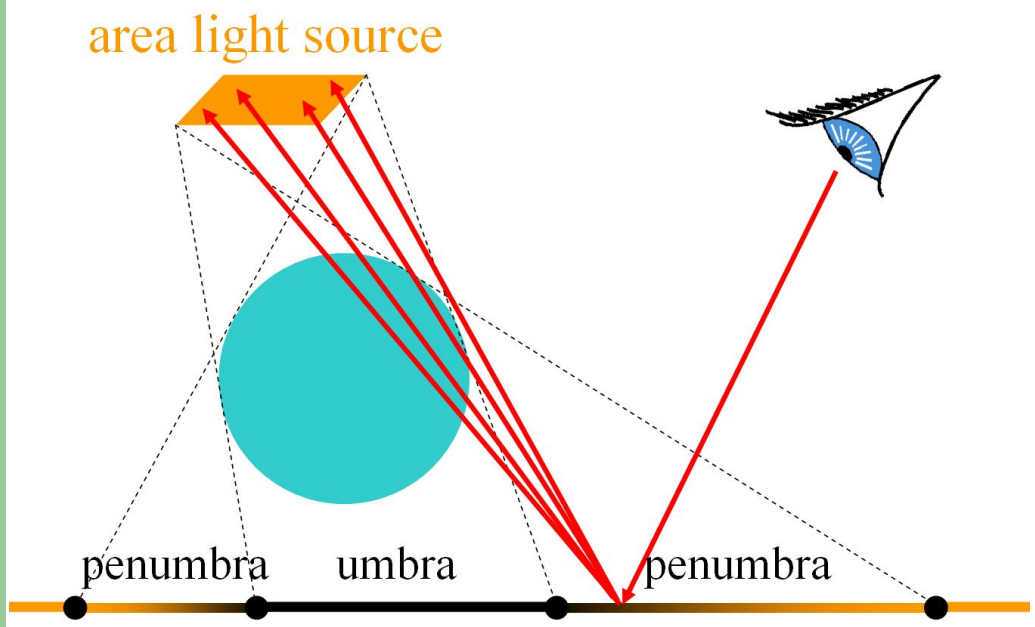
http://www.pa.uky.edu/~sciworks/light/preview/bulb2.htm

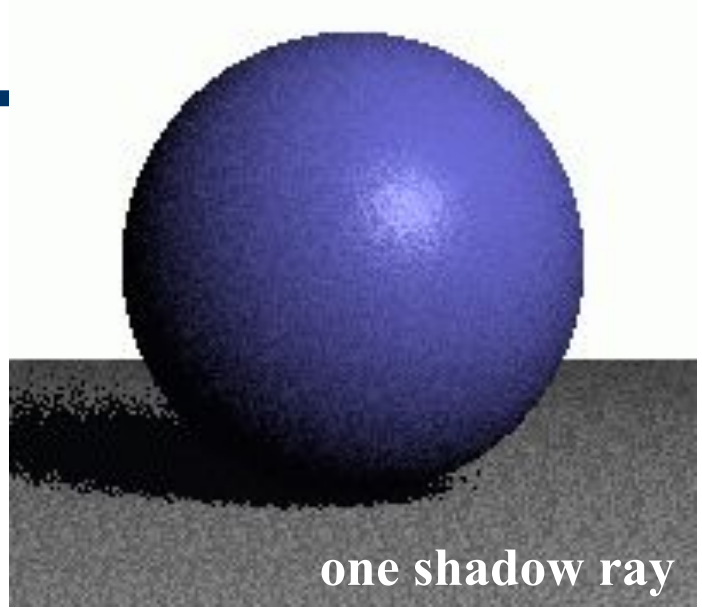
Area Lights

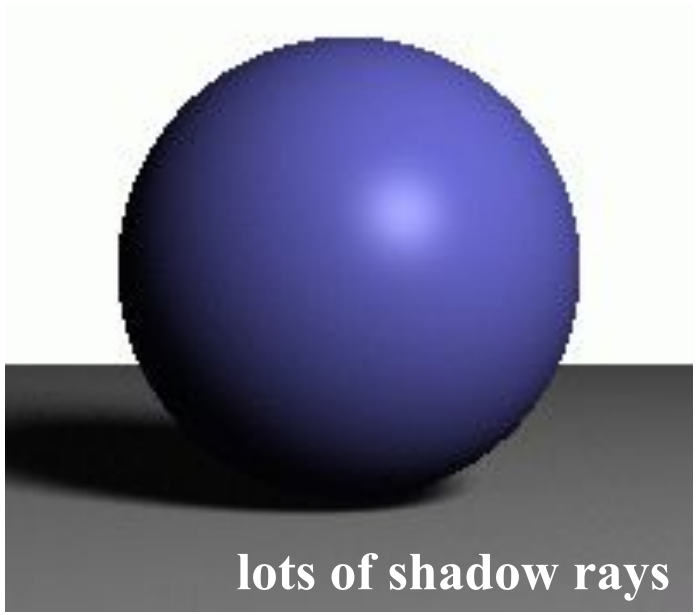
- The soft shadow technique enables us to define lights in a much more complex way than we have previously
- We can now use any geometry to define a light, including triangles, patches, spheres, etc.
- To determine the incident light, we shoot several rays towards the light source, distributed across the surface and weighted according to the surface area of the sample and the direction of the average normal
- Larger light sources create softer, diffuse shadows, while smaller light sources cause sharp, harsh shadows
- Larger light sources also require more rays to adequately sample the shadows, making area lights a lot more expensive than point lights. Inadequate sampling of the light source can cause noise patterns to appear in the penumbra region, known as *shadow aliasing*

Soft Shadows

- multiple shadow rays to sample area light source

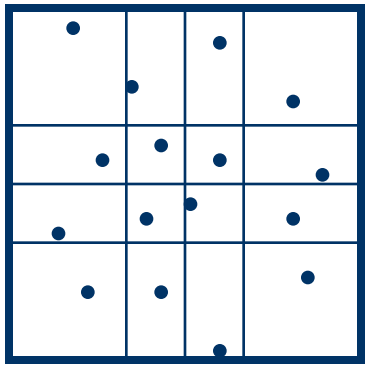






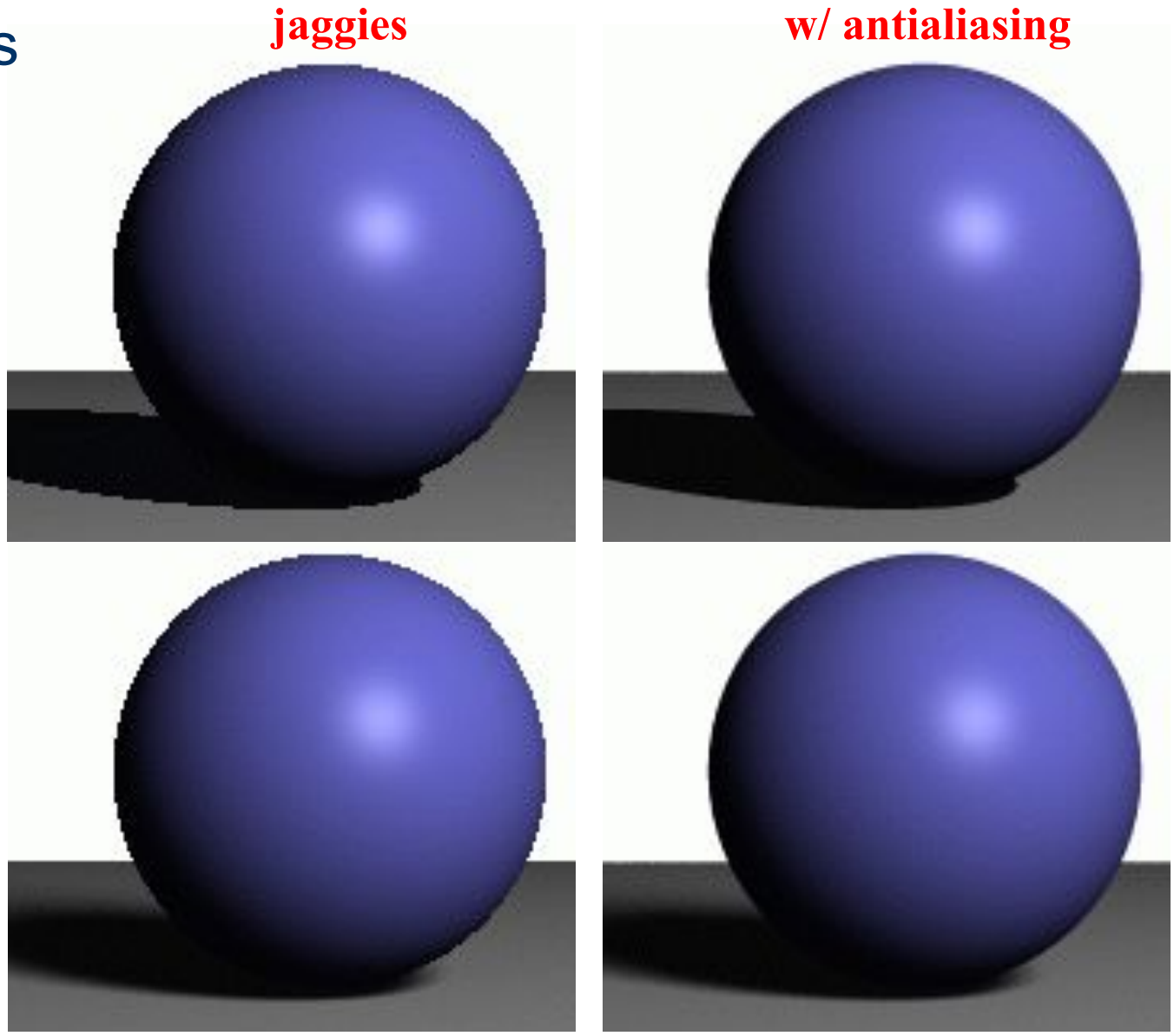
Antialiasing

- For good quality antialiasing, we will can shoot several primary rays for each pixel
- several different antialiasing, area-weighted, jittered Gaussian distribution was chosen as one of the best overall sampling schemes
- As we have complete control over the direction of every ray, we can use any sampling pattern we choose, as well as any number of rays we choose



Antialiasing – Supersampling

- multiple rays per pixel

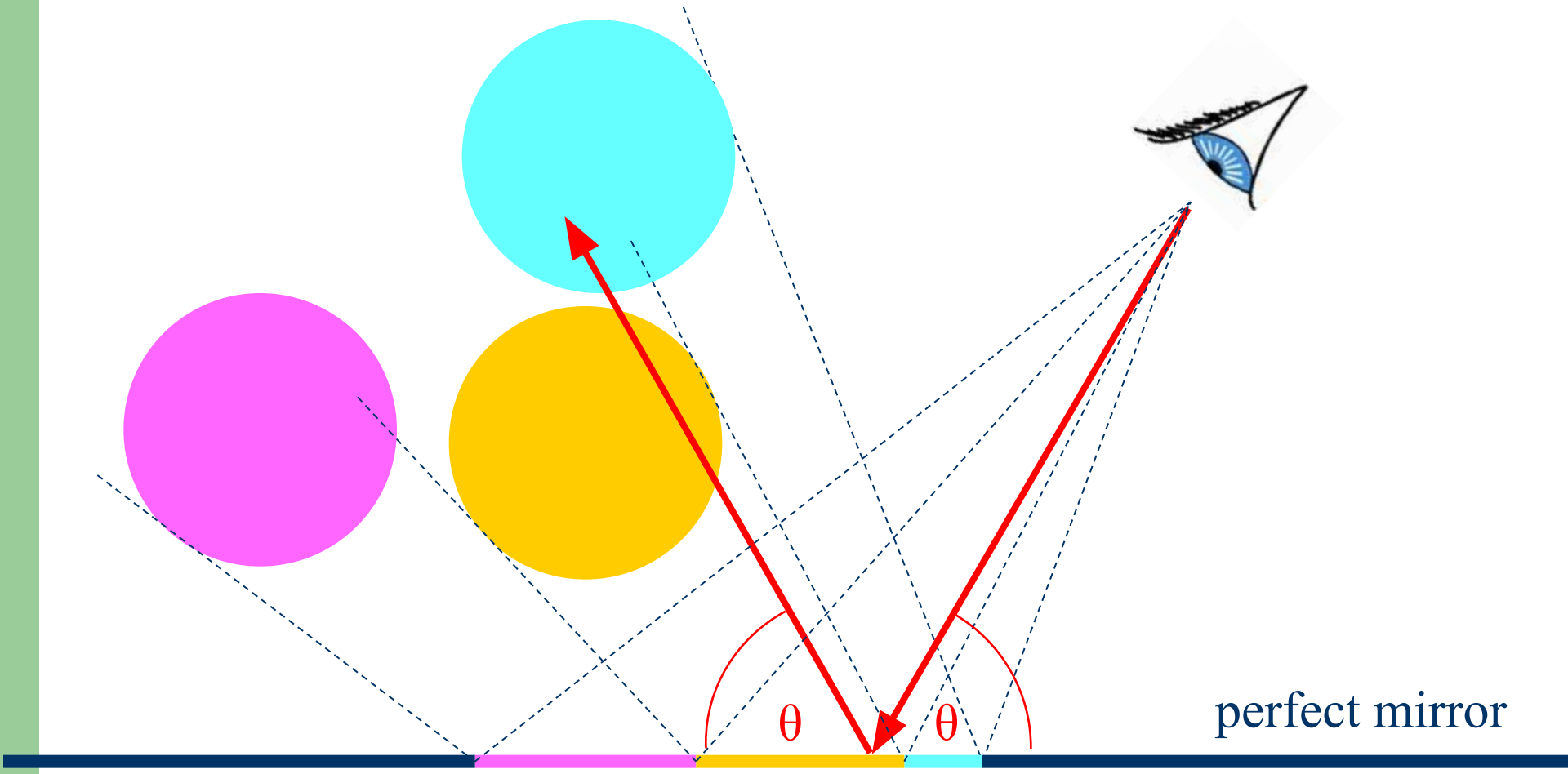


Blurry Reflections

- We can render blurry or *glossy* reflections by creating several reflection rays instead of just one
- The rays can be distributed around the ideal reflection direction
- Blurry surfaces will causes a wider distribution (and require more rays), while more polished surfaces will have a narrow distribution
- The same concept can apply to refraction in order to achieve rendering of unpolished glass

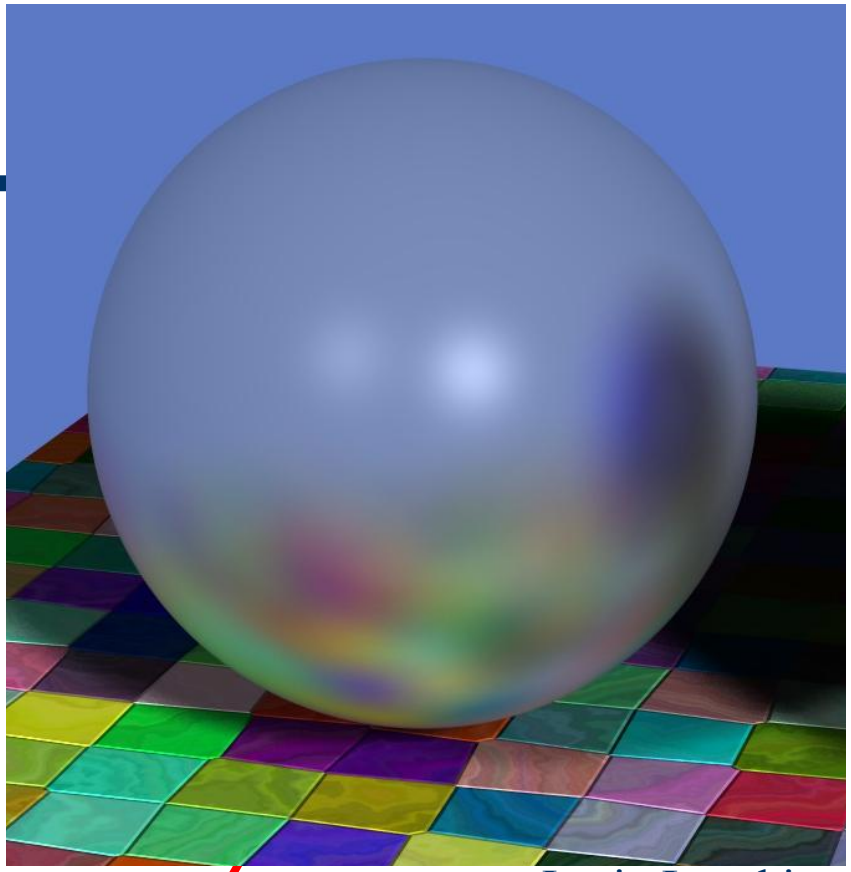
Reflection

- one reflection ray per intersection

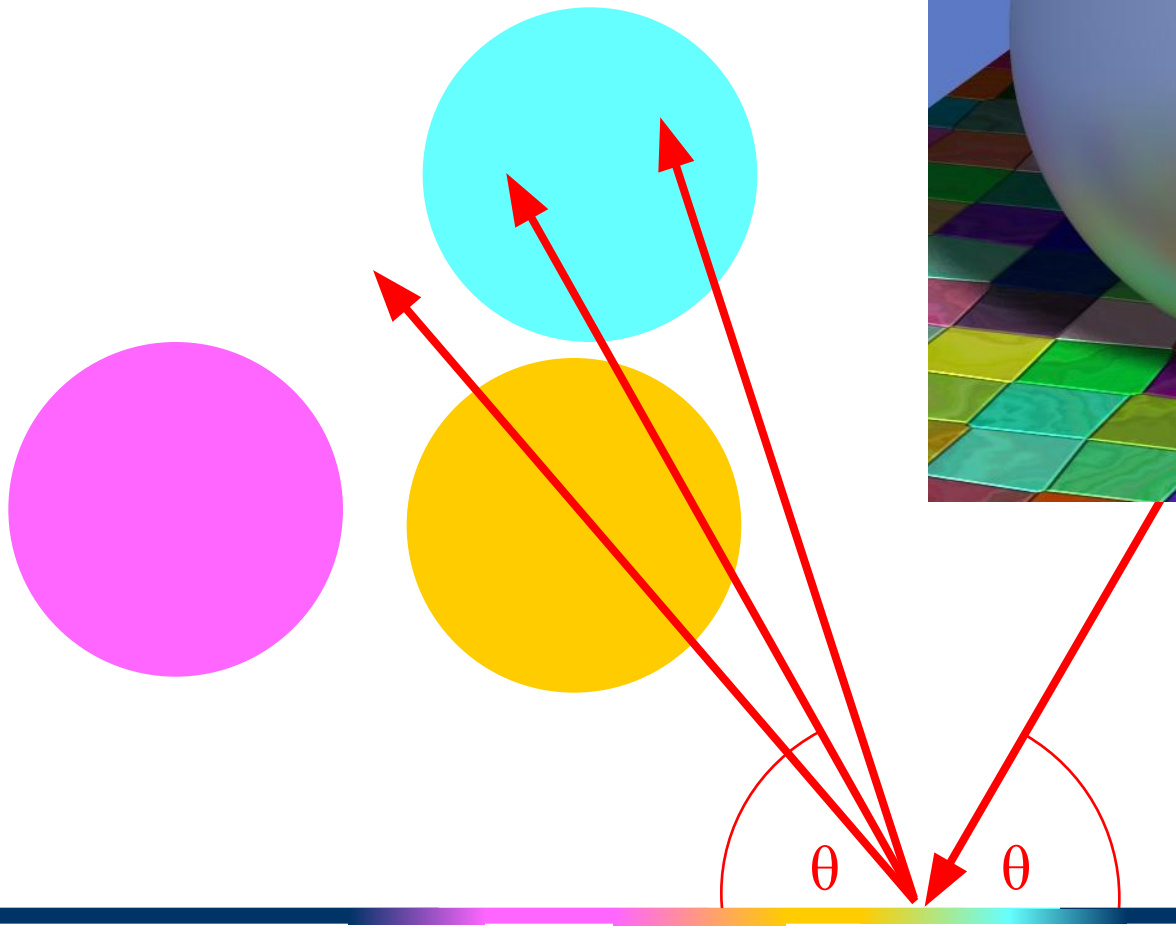


Glossy Reflection

- multiple reflection rays



Justin Legakis



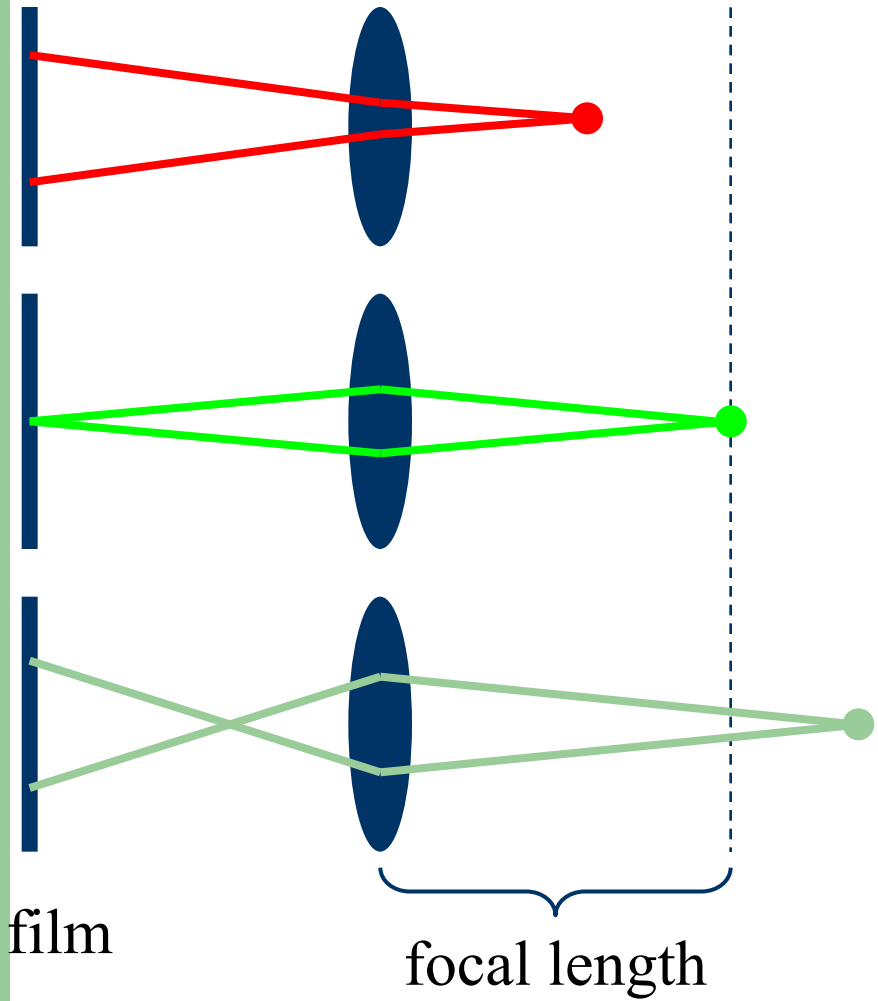
polished surface

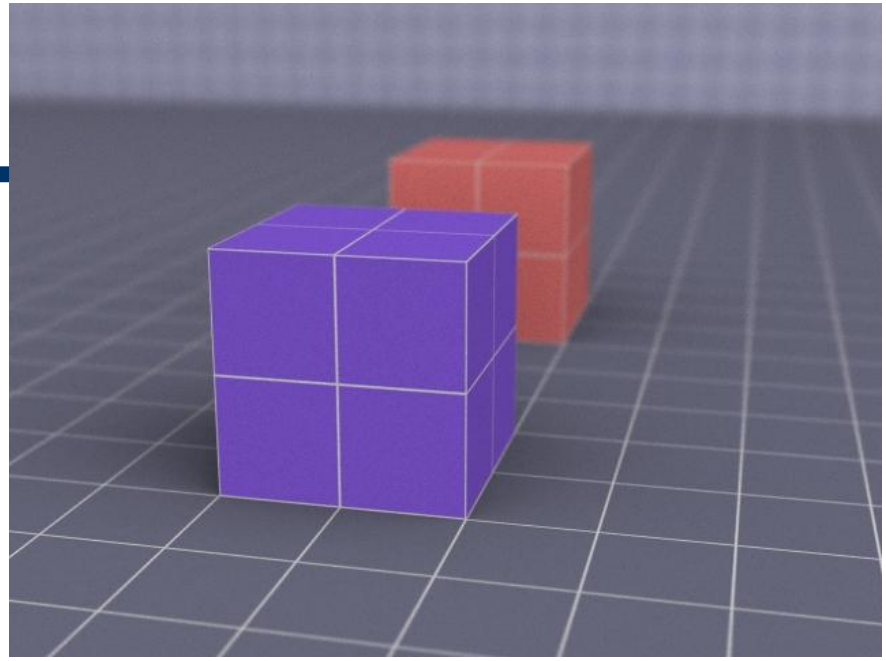
Depth of Field

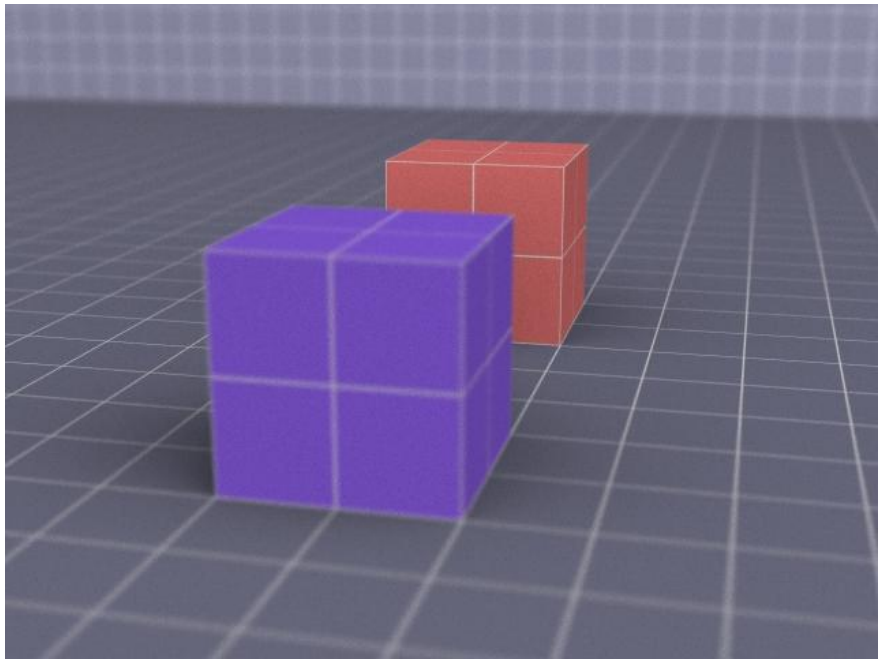
- The blurring caused by a camera lens being out of focus is due to the lens' limited *depth of field*
- In computer graphics, the term 'depth of field' usually refers to the general process of rendering images that include a camera blurring effect
- A lens will typically be set to focus on objects at some distance away, known as the *focal distance*
- Objects closer or farther than the focal distance will be blurry, and the blurriness increases with the distance to the *focal plane*
- Depth of field can be rendered with distribution ray tracing by distributing the primary rays shot from the camera
- Rays area distributed across a virtual *aperture*, which represents the (usually circular) opening of the lens
- The larger the aperture, the more pronounced the blurring effect will be. A *pinhole* camera has an aperture size of 0, and therefore, will not have any blurring due to depth of field

Depth of Field

- multiple rays per pixel







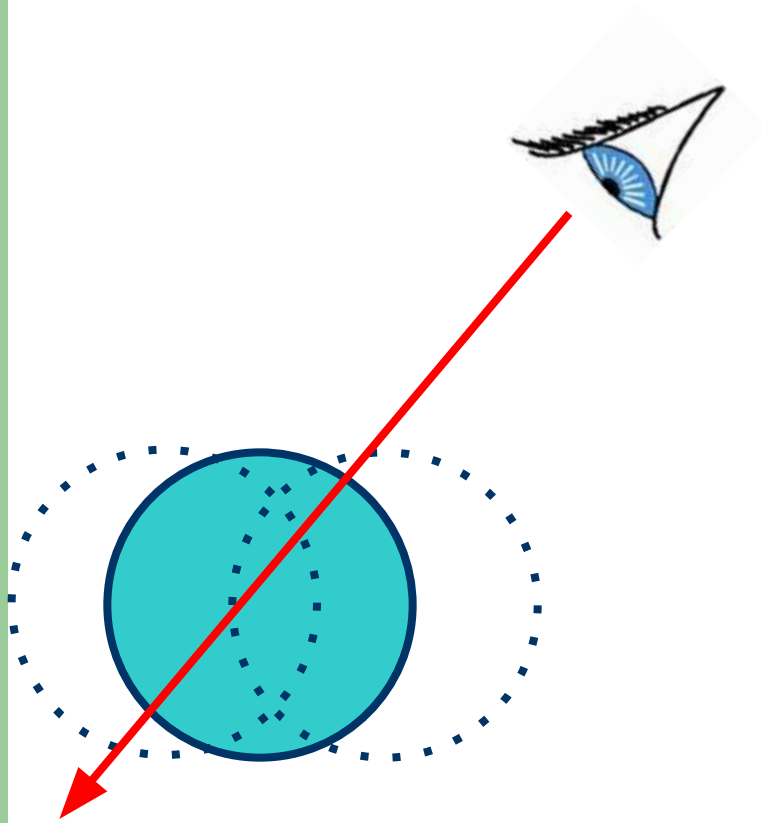
Justin Legakis

Motion Blur

- As if soft shadows, antialiasing, depth of field, and blurry reflections and refractions weren't enough… we can also use distribution ray tracing to include the effect from motion blur
- We do this by distributing rays in time
- Therefore, we need to know the starting and ending transformations for all of the objects in the scene (and the lights & camera)
- In fact, we can even consider objects moving along more complex paths, so we can motion blur bounces as well

Motion Blur

- Sample objects temporally





Rob Cook

Distribution Ray Tracing

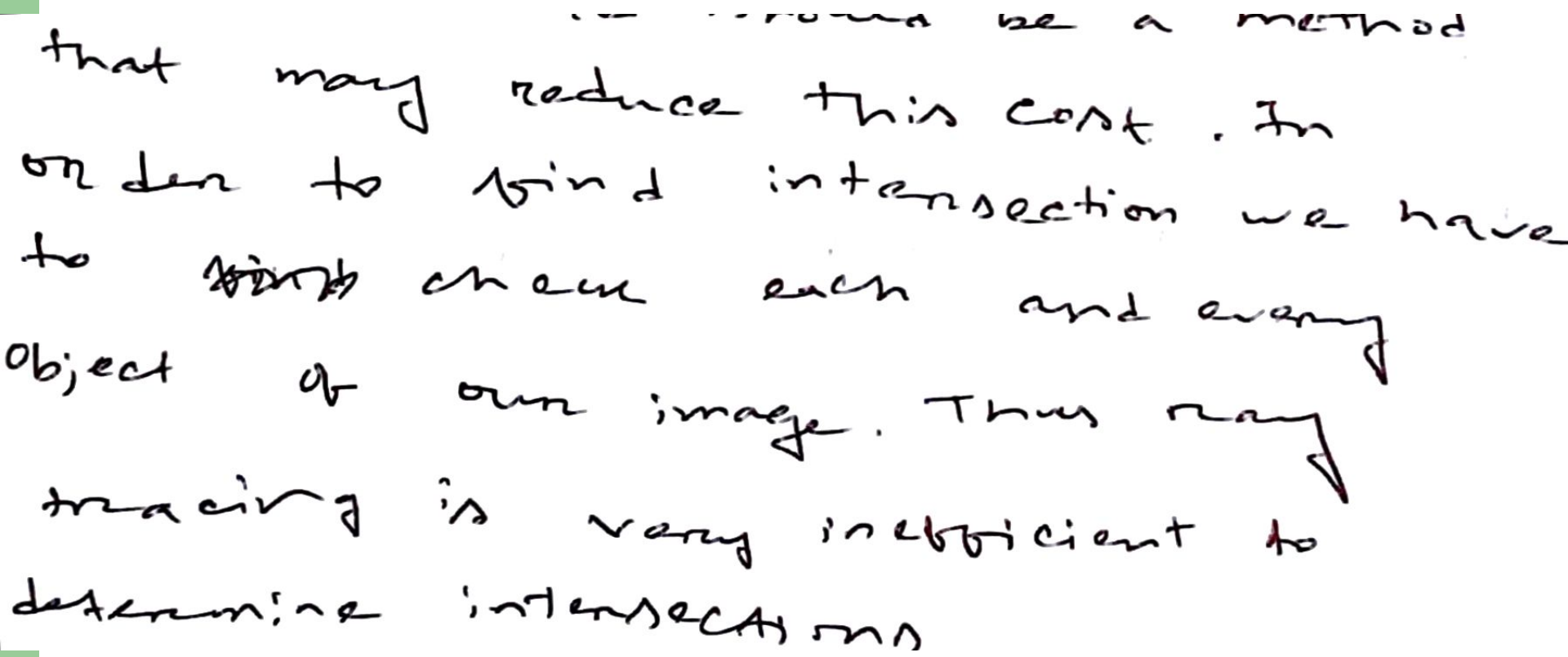
- Ray tracing had a big impact on computer graphics in 1980 with the first images of accurate reflections and refractions from curved surfaces
- Distribution ray tracing had an even bigger impact in 1984, as it re-affirmed the power of the basic ray tracing technique and added a whole bunch of sophisticated effects, all within a consistent framework
- Previously, techniques such as depth of field, motion blur, soft shadows, etc., had only been achieved individually and by using a variety of complex, hacky algorithms

Distribution Ray Tracing

- If ray tracing is slow, then distribution ray tracing must be considerably slower
- Now, instead of one or two splits per level in our recursion, we are have to shoot dozens or even hundreds of rays to achieve some of these effects
- This can cause an exponential expansion in the number of rays
- The good news is that we can combine these features so that we still only need to shoot a small number of primary rays per pixel
- For example, we can shoot 16 rays in a 4x4 antialiasing pattern, where each ray has a random distribution in time and in the camera aperture
- Each of these rays only needs to spawn a few reflection or shadow rays, as the results will be blended with 15 other samples
- Still, we end up with lots and lots of rays and potential for exponential problems in scenes with a lot of soft or blurry features
- This problem is at least partially addressed with *path tracing* which is one of the techniques for *global illumination* that we will see in the next lecture

c) In ray tracing a large computational cost is associated with determining ray-object intersections. Suggest a method that may reduce this cost. (10)

In ray tracing a large computational cost is associated with determining ray-object injections. It requires a lot of computational complexity to do ray tracing. Thus ray tracing is very much computationally inefficient and thus there should be a method that



- Don’t answer like this, you will carry only zero marks

- https://www.youtube.com/watch?v=iOlehM5kNSk